SCHAEFFLER × TUM

How Lean Can We Go?

Optimization of Artificial Neural Networks

Constructive Topology Search versus Pruning for Compact Regression Models in Electric-Motor Thermal Monitoring

Institution	TUM Department of Mathematics
Course	CIT413058
Supervisors	PD Dr. T. Köppl, M.Sc. F. Holzberger
Industry partner	Schaeffler AG

Authors

Yanqin Li Zunaira Haider Aryan Auti

Simon Giuliani Marko Prohaska

July 28, 2026

Abstract

Permanent-magnet synchronous motors (PMSMs) fail at the stator winding when it overheats, yet placing a temperature sensor there is costly, fragile, or mechanically impossible. The standard remedy is a data-driven estimator that infers winding temperature from signals the drive already measures. Because that estimator must run on cheap embedded hardware in millions of units, every parameter it carries is a recurring cost — which raises the question this report is named after: *how lean can such a network actually be?*

We compare the two opposing routes to a compact network. The **subtractive** route trains a large network and removes what it can spare; we implement ten pruning criteria spanning magnitude, first- and second-order saliency (Taylor-FO, Optimal Brain Damage, Optimal Brain Surgeon), neuron-level scores (structured, activation, redundancy), sparsity-at-initialisation (SNIP, IMP+Rewind) and low-rank factorisation. The **constructive** route grows a network from a minimal seed, adding and removing neurons and layers under a Lagrangian saliency estimate and Gauss–Newton retraining; we implement the ADAGOS algorithm of US 12,124,779 B2 and follow it faithfully.

We evaluate both on a controlled synthetic quadratic family (input dimension $d \in \{1, 2, 3\}$, clean and noisy) and on the Kaggle Electric Motor Temperature PMSM benchmark (1.33 M samples, 69 drive sessions). Four findings emerge. **(i) Noise, not input dimension, governs leanness and prunability**: on noisy data the search converges to 7–15 parameters and pruning reaches 50–70% sparsity, while on clean data the search grows without brake and almost nothing is prunable within the error budget. **(ii) No pruning criterion wins everywhere** — loss-aware criteria (OBS/OBD/Taylor-FO) dominate the small synthetic networks, whereas structural and IMP+Rewind criteria dominate the large motor networks. **(iii) On the real motor task the constructed network is competitive with a fully tuned baseline while being 50–500× smaller**: ADAGOS reaches 0.97 °C MAE with 133 parameters against 1.03 °C with 69,121 for the tuned MLP, and construct-then-prune reaches 0.79 °C with 75 parameters on the feature-rich variant. **(iv) A dynamic thermal twin must be pruned on free-running rollout stability, not one-step error** — a criterion that looks benign at one-step horizon diverges to 103 °C under rollout.

Taken together, the results support a practical recipe: *construct, then prune*, with the operating point chosen on the metric the deployed system actually experiences.

Contents

1 Introduction **4**
1.1 Industrial motivation . . . 4
1.2 Two opposing philosophies . . . 4
1.3 Scope and contributions . . . 5
1.4 Structure of this report . . . 5

2 Background and Notation **5**
2.1 The multilayer perceptron . . . 5
2.2 Why architecture selection is the hard part . . . 6
2.3 Activation function . . . 6
2.4 Metrics and reporting conventions . . . 7

3 Pruning: The Subtractive Route **7**
3.1 Granularity: weights versus neurons . . . 8
3.2 Post-training weight criteria . . . 8
3.2.1 Magnitude . . . 8
3.2.2 Taylor first-order (Taylor-FO) . . . 9
3.2.3 Optimal Brain Damage (OBD) . . . 9
3.2.4 Optimal Brain Surgeon (OBS) . . . 9
3.3 Post-training neuron criteria . . . 9
3.4 Sparsity at and during training . . . 10
3.5 Low-rank factorisation . . . 10
3.6 Common experimental protocol . . . 10

4 Adagos: The Constructive Route **10**
4.1 Overview . . . 11
4.2 The objective, and where parsimony comes from . . . 11
4.3 The Lagrangian saliency estimate . . . 12
4.4 Gauss–Newton retraining . . . 12
4.5 Candidate generation: every edit begins as a no-op . . . 13
4.6 The acceptance gate . . . 13
4.7 Implementation notes . . . 14
4.8 Relation to prior art . . . 14

5 Experiments on Synthetic Data **15**
5.1 Datasets . . . 15
5.2 Protocol . . . 16
5.3 Pruning results . . . 17
5.4 Constructive results . . . 17
5.5 Four findings . . . 18

6 The Electric-Motor Task **21**
6.1 Dataset . . . 21
6.2 Splitting: why profiles, not samples . . . 22
6.3 Feature engineering . . . 23
6.4 Hyperparameter search and final training . . . 23
6.5 Baseline quality . . . 24
6.6 Static estimator: pruning . . . 25
6.6.1 Why OBS collapses onto OBD . . . 25
6.6.2 The engineered baselines compress; the raw one does not . . . 26

6.7 Static estimator: construction . . . 27
6.8 Head-to-head . . . 28
6.9 Dynamic thermal twin . . . 29
6.9.1 Prune on rollout, not on one-step error . . . 29

7 Evaluation **31**
7.1 Accuracy per parameter . . . 31
7.2 Cost of producing the model . . . 31
7.3 Robustness . . . 32
7.4 What worked . . . 32
7.5 What did not work, and why . . . 33
7.6 A deployment recipe . . . 33

8 Next Steps **33**
8.1 Improving the constructive algorithm . . . 33
8.2 Extending the experimental programme . . . 34
8.3 Toward deployment . . . 35
8.4 Priority . . . 35

9 Conclusion **35**

A Notation **38**

B Experimental configuration **39**

C Reproducibility **39**

D Figure regeneration **40**

1 Introduction

1.1 Industrial motivation

Permanent-magnet synchronous motors (PMSMs) are the workhorse of modern e-mobility drivetrains. Their dominant thermal failure mode is degradation of the insulation on the *stator winding*, which occurs when the winding exceeds its rated temperature. Protecting the motor therefore requires knowing the winding temperature continuously and in real time.

Measuring it directly is the obvious solution and the impractical one. The winding is embedded in the stator slots, rotating machinery and potting compound surround it, and a thermocouple placed there is expensive to install, prone to failure, and in many designs simply cannot be routed. Industry practice is consequently to *estimate* the winding temperature from quantities the inverter already measures — d/q-axis voltages and currents, motor speed, torque, and ambient and coolant temperatures.

That estimator is a piece of embedded software replicated across millions of motors. Its parameter count is not an academic quantity: it sets flash footprint, inference latency, and energy per evaluation on a microcontroller whose budget is measured in kilobytes. A model that is 500× smaller at equal accuracy is a direct, recurring reduction in bill-of-materials cost. This gives the project its guiding question:

> **How lean can we go?** Given a target accuracy on a real motor-thermal regression task, what is the smallest neural network that attains it, and which family of methods finds that network most reliably?

1.2 Two opposing philosophies

There are two ways to arrive at a compact network, and they are exact opposites (fig. 1).

Subtractive — pruning.
Train a network that is deliberately larger than necessary, score every weight or neuron by some notion of importance, remove the least important, and fine-tune. The large model is a scaffold: it makes optimisation easy, and the scaffold is then dismantled. This is the mainstream approach and carries thirty-five years of literature, from Optimal Brain Damage [7] to the Lottery Ticket Hypothesis [3].

Constructive — topology search.
Begin from the smallest network that can express anything at all, and add structure only where the data proves it is needed, removing structure when it stops paying for itself. The network is never larger than the evidence justifies. We study the ADAGOS algorithm patented as US 12,124,779 B2 [2] as the concrete representative of this family.

The two routes make opposite bets. Pruning bets that a good small network is easiest to find *inside* a good large one — that over-parameterisation buys a benign optimisation landscape. Construction bets that the large network is an unnecessary detour, and that greedy, error-driven growth reaches an equally good small network without ever paying for the large one, and without the architecture search (or hyper-parameter optimisation) that choosing the large one requires in the first place.

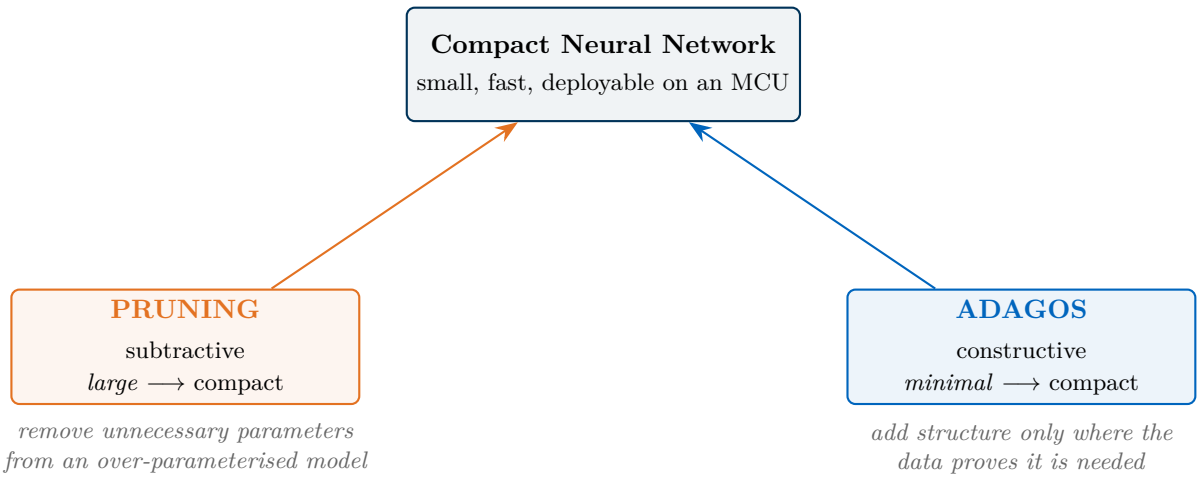


Figure 1: The two strategies pursue an identical goal from opposite directions. Pruning starts too big and removes; ADAGOS starts too small and adds. The central empirical question of this report is which route reaches the better operating point, and at what cost.

1.3 Scope and contributions

We evaluate both families under a single, shared protocol — identical data splits, identical error budgets, identical reporting — first on a controlled synthetic family where the ground-truth function is known, then on a real PMSM benchmark. Our contributions are:

- **C1.** A faithful implementation of the ADAGOS constructive algorithm (section 4), including the cross-Lagrangian saliency estimate, the Gauss–Newton/Levenberg–Marquardt retraining step, and identity-preserving candidate generation, verified claim-by-claim against the granted patent.
- **C2.** A ten-method pruning suite (section 3) under one interface, spanning weight-level, neuron-level, at-initialisation and low-rank criteria, each with a masked LBFGS polish so that comparisons are not confounded by under-converged fine-tuning.
- **C3.** A controlled synthetic study (section 5) that isolates the effect of *noise* from that of *input dimension*, yielding the report's central mechanistic finding.
- **C4.** A real-motor study (section 6) with leakage-free profile-level splits, multi-scale EWMA feature engineering, a literature-competitive baseline, and both a static estimator and a free-running dynamic thermal twin.
- **C5.** An evaluation and outlook (sections 7 and 8) that turns the results into a deployment recipe and a concrete research agenda.

1.4 Structure of this report

Section 2 fixes notation and reviews the multilayer perceptron. Section 3 develops the pruning suite and derives each saliency criterion. Section 4 presents the constructive algorithm. Section 5 reports the synthetic experiments and the four findings. Section 6 reports the PMSM study. Section 7 evaluates both families against each other, and section 8 sets out next steps.

2 Background and Notation

2.1 The multilayer perceptron

Every model in this report is a feed-forward multilayer perceptron (MLP): an alternating composition of affine maps and a scalar non-linearity. For an input $\boldsymbol{x} \in \mathbb{R}^d$, weights $\mathbf{W}^{(\ell)}$,

biases $\boldsymbol{b}^{(\ell)}$ and activation φ,

$$\boldsymbol{h}^{(1)} = \varphi\Big(\mathbf{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}\Big), \tag{1}$$

$$\boldsymbol{h}^{(\ell)} = \varphi\Big(\mathbf{W}^{(\ell)}\boldsymbol{h}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}\Big), \qquad \ell = 2, \ldots, L-1, \tag{2}$$

$$\hat{y} = \mathbf{W}^{(L)}\boldsymbol{h}^{(L-1)} + \boldsymbol{b}^{(L)}. \tag{3}$$

The output layer is deliberately linear: the targets are physical temperatures in °C, and a squashing non-linearity on the output would bound the reachable range. The parameter count that we minimise throughout is

$$P \;=\; \sum_{\ell=1}^{L} (n_{\ell-1} + 1)\; n_{\ell}, \tag{4}$$

with n_ℓ the width of layer ℓ, $n_0 = d$ and $n_L = 1$.

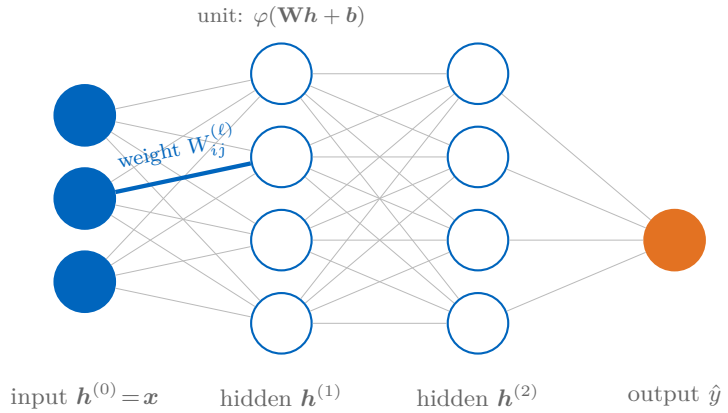


Figure 2: An MLP with two hidden layers. Learned quantities are the weights and biases; the *hyper-parameters* — depth L, widths n_ℓ, activation φ, optimiser and learning rate — must be fixed in advance. Everything in this report is ultimately about who chooses those hyper-parameters: a human running a search, or the algorithm itself.

2.2 Why architecture selection is the hard part

The Universal Approximation Theorem guarantees that a single hidden layer of sufficient width can approximate any continuous function on a compact domain to arbitrary accuracy. It is silent on the two questions an engineer actually has: *how wide*, and *how do I find those weights*. In practice the widths and depth are chosen by hyper-parameter optimisation (HPO) — a search over a discrete grid, evaluated by training a network per configuration. This is where the cost sits: our tuned synthetic baselines each consumed up to 240 Optuna trials, and each trial is a full training run.

Both method families in this report can be read as automating that choice. Pruning automates it *downward* from a configuration the human still had to pick; construction automates it *upward* from a seed that requires no choice at all. This asymmetry — who pays for architecture search — recurs throughout section 7.

2.3 Activation function

Both tracks use the hyperbolic tangent, $\varphi(z) = \tanh(z)$, on hidden layers. This is a deliberate constraint inherited from the patent, whose specification requires a sigmoidal activation, and it is applied to the pruning track as well so that the comparison is not confounded by activation choice. Two consequences matter later:

- tanh is smooth and bounded, so second-order saliency criteria (sections 3.2.3 and 3.2.4) are well defined and the Gauss–Newton approximation of section 4.4 is well behaved.
- tanh *saturates*. For inputs of large magnitude the gradient vanishes, which limits how much a single added neuron can contribute on feature-rich, wide-input problems. We return to this in section 7.5, where it explains why the constructive search stalls on the 81-input motor variant.

2.4 Metrics and reporting conventions

Error.
On synthetic data we report mean squared error (MSE) on a held-out test split. On the motor data we report mean absolute error (MAE) and root mean squared error (RMSE) in °C, because degrees are the unit in which an engineer specifies a thermal margin.

Model selection is on validation, reporting on test.
Every architecture and sparsity level in this report is chosen using a validation split and reported on a test split that played no part in selection. On the motor data the split is at the level of whole drive *profiles*, never individual samples, since consecutive samples of one session are strongly correlated and a random split would leak.

Error budget.
When we say a pruning level is "within budget" we mean its validation error stays within a fixed multiplicative tolerance of the unpruned baseline's. Comparisons of "how sparse can we get" are always at equal budget.

Ratios.
MSE×base denotes pruned or constructed error divided by the corresponding baseline error: 1.00× is parity, below 1 is better than baseline.

3 Pruning: The Subtractive Route

Pruning removes the least important weights or neurons from a trained network, trading a controlled amount of accuracy for a reduction in size. All methods share a common skeleton — *score, remove, recover* — and differ only in how importance is scored and when the scoring happens.

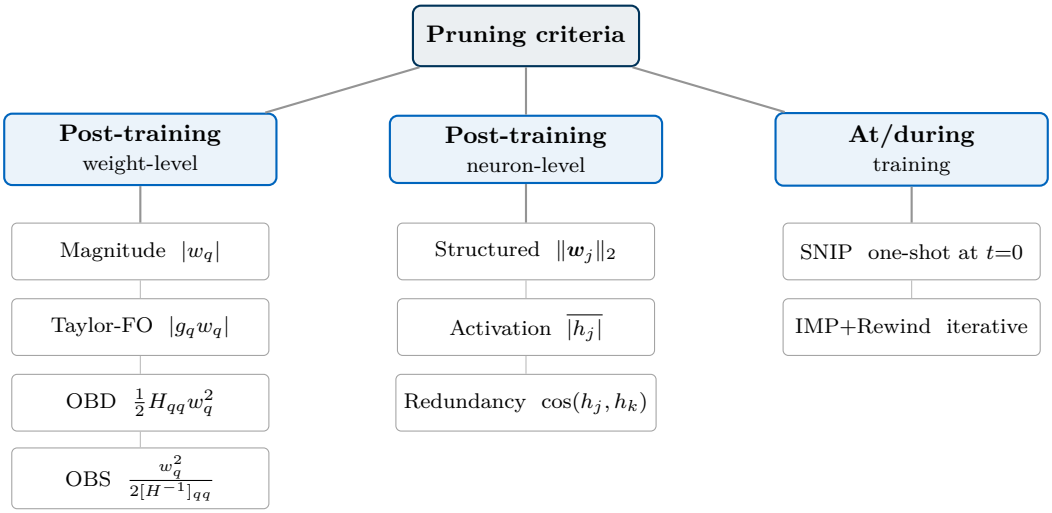
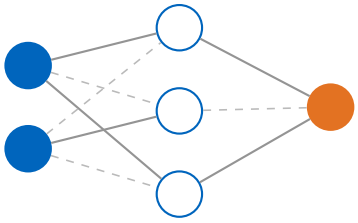
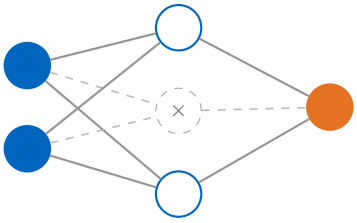


Figure 3: Taxonomy of the ten pruning criteria implemented. The families differ in *granularity* (individual weights versus whole units) and in *when* importance is measured (after training, or before optimisation has begun). A tenth method, low-rank factorisation, replaces $\mathbf{W}$ by a truncated SVD and does not fit this tree.

3.1 Granularity: weights versus neurons



(a) Weight-based. Individual edges are removed; every neuron survives. Produces an *unstructured* sparse matrix.



(b) Neuron-based. A whole unit and all its edges go at once. Produces a genuinely *smaller dense* matrix.

Figure 4: Granularity determines whether the saving is realisable on real hardware. Unstructured sparsity needs sparse-matrix support to pay off; removing a neuron shrinks the dense shapes and always pays off. This distinction becomes decisive in section 6.6.

3.2 Post-training weight criteria

All post-training criteria start from the same second-order Taylor expansion of the loss L around the trained weights under a perturbation $\delta\boldsymbol{w}$:

$$\Delta L \;\approx\; \boldsymbol{g}^{\top}\delta\boldsymbol{w} \;+\; \tfrac{1}{2}\,\delta\boldsymbol{w}^{\top}\mathbf{H}\,\delta\boldsymbol{w}, \qquad \boldsymbol{g} = \nabla_{\boldsymbol{w}} L, \quad \mathbf{H} = \nabla^2_{\boldsymbol{w}} L. \tag{5}$$

Removing weight q means $\delta\boldsymbol{w} = -w_q \boldsymbol{e}_q$. The criteria are distinguished purely by which terms of eq. (5) they keep.

3.2.1 Magnitude

The simplest possible assumption: small weights matter little.

$$s_q^{\mathrm{MAG}} = |w_q|. \tag{6}$$

It uses no loss information at all, which makes it free to compute and blind to the possibility that a small weight sits on a very sensitive direction.

3.2.2 Taylor first-order (Taylor-FO)

Keep only the gradient term of eq. (5). Substituting $\delta \boldsymbol{w} = -w_q \boldsymbol{e}_q$ gives $\Delta L \approx -g_q w_q$, hence

$$s_q^{\mathrm{TFO}} = |g_q \, w_q| \, . \tag{7}$$

Note the tension: at a true local optimum $\boldsymbol{g} \to \mathbf{0}$ and this score degenerates. It is informative in practice only because training stops near, not at, a stationary point.

3.2.3 Optimal Brain Damage (OBD)

Assume the network *is* at a local optimum, so $\boldsymbol{g} \approx \mathbf{0}$ and the first-order term vanishes; then approximate the Hessian by its diagonal, $\mathbf{H} \approx \mathrm{diag}(\mathbf{H})$. The quadratic term survives alone:

$$\Delta L \approx \tfrac{1}{2} H_{qq} w_q^2 \qquad \Longrightarrow \qquad s_q^{\mathrm{OBD}} = \tfrac{1}{2} H_{qq} \, w_q^2 . \tag{8}$$

This is magnitude pruning *reweighted by local curvature*: a small weight on a sharply curved direction is now correctly protected.

3.2.4 Optimal Brain Surgeon (OBS)

OBD's diagonal assumption ignores that removing w_q can be *compensated* by adjusting the surviving weights. OBS keeps the full Hessian and solves for the optimal compensation explicitly:

$$\min_{\delta \boldsymbol{w}} \ \tfrac{1}{2} \delta \boldsymbol{w}^\top \mathbf{H} \, \delta \boldsymbol{w} \quad \text{s.t.} \quad \boldsymbol{e}_q^\top \delta \boldsymbol{w} + w_q = 0. \tag{9}$$

With Lagrangian $\ell = \tfrac{1}{2} \delta \boldsymbol{w}^\top \mathbf{H} \delta \boldsymbol{w} + \lambda (\boldsymbol{e}_q^\top \delta \boldsymbol{w} + w_q)$, stationarity gives $\mathbf{H} \delta \boldsymbol{w} + \lambda \boldsymbol{e}_q = 0$, so $\delta \boldsymbol{w} = -\lambda \mathbf{H}^{-1} \boldsymbol{e}_q$. Enforcing the constraint yields $\lambda = w_q / [\mathbf{H}^{-1}]_{qq}$, and therefore

$$\delta \boldsymbol{w}^\star = -\frac{w_q}{[\mathbf{H}^{-1}]_{qq}} \, \mathbf{H}^{-1} \boldsymbol{e}_q, \qquad s_q^{\mathrm{OBS}} = \frac{w_q^2}{2 \, [\mathbf{H}^{-1}]_{qq}} . \tag{10}$$

OBS is theoretically the strongest of the four: it accounts for interactions between all weights and it *updates* the survivors rather than merely deleting. Its cost is the inverse Hessian, an $n \times n$ object. This is not a footnote — it decides the method's fate on the motor networks (section 6.6.1).

3.3 Post-training neuron criteria

Structured.
Score neuron j by the ℓ_2 norm of its outgoing weights, $s_j = \|\boldsymbol{w}_j\|_2 = \big(\sum_i w_{ji}^2\big)^{1/2}$. A neuron whose outgoing weights are collectively small cannot influence the output much.

Activation.
Score by mean absolute activation over the dataset, $s_j = \frac{1}{N} \sum_{i=1}^{N} |h_j(\boldsymbol{x}_i)|$. A unit that is consistently near zero contributes nothing regardless of its weights. Unlike the other criteria this one is *data-dependent*.

Redundancy.
Score by *duplication* rather than smallness. With $\mathrm{sim}(j,k) = \frac{\boldsymbol{h}_j^\top \boldsymbol{h}_k}{\|\boldsymbol{h}_j\| \|\boldsymbol{h}_k\|}$, any neuron with $\max_{k \neq j} \mathrm{sim}(j,k) > \tau$ is a near-duplicate of another and one of the pair is removed. This targets a failure mode the magnitude-like criteria structurally cannot see: two large, important, and identical units.

3.4 Sparsity at and during training

The criteria above all presuppose a trained network. Two influential methods do not.

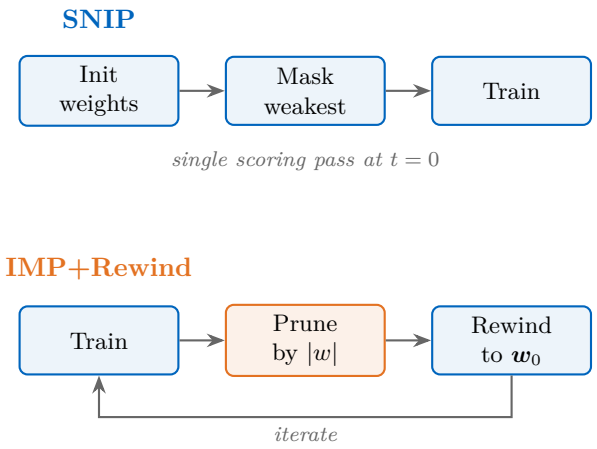


Figure 5: SNIP scores connections *once*, at initialisation, using $s_q = |w_q \, \partial L/\partial w_q|$ evaluated on the random initial weights, then trains the surviving subnetwork. IMP+Rewind alternates train–prune–rewind, resetting survivors to their *initial* values each round — the Lottery Ticket procedure. Both therefore depend on the random initialisation in a way the post-training criteria do not.

3.5 Low-rank factorisation

The tenth method is structurally different: rather than zeroing entries it replaces a weight matrix by a low-rank product. Taking the SVD $\mathbf{W} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ and truncating to the top r singular values gives $\mathbf{W} \approx \mathbf{U}_r\mathbf{\Sigma}_r\mathbf{V}_r^\top$, which costs $r(n_{\text{in}} + n_{\text{out}})$ parameters instead of $n_{\text{in}}n_{\text{out}}$ and is a saving whenever $r < \frac{n_{\text{in}}n_{\text{out}}}{n_{\text{in}}+n_{\text{out}}}$. It preserves dense arithmetic, so the saving is always realisable — but on the narrow networks studied here the break-even rank is small and the method rarely finds a useful operating point.

3.6 Common experimental protocol

To make the ten criteria comparable, all share one pipeline:

1. Train (or, for SNIP/IMP, initialise) the baseline network.
2. Score all prunable parameters by the criterion under test.
3. Remove the lowest-scoring fraction to hit a target sparsity, applied as a binary mask so that removed parameters are held at exactly zero.
4. **Recover:** fine-tune the survivors — an Adam phase followed by a masked strong-Wolfe LBFGS polish. The polish matters: without a second-order finishing step, differences between criteria are contaminated by differences in how well each pruned network was re-converged, and weaker criteria are unfairly penalised.
5. Evaluate on validation to select the operating point; report on test.

Sparsity is swept over $\{10, 20, \ldots, 90, 95\}\%$ so that each criterion is characterised by a *curve*, not a single point — the shape of that curve turns out to be as informative as its minimum.

4 Adagos: The Constructive Route

Adagos is the constructive counterpart: instead of dismantling a large network, it *assembles* a small one. The algorithm is the subject of US 12,124,779 B2 [2], and our implementation fol-

lows the granted claims: a pure sum-of-squares objective, a sigmoidal activation, a Lagrangian-multiplier saliency estimate for ranking candidate topology edits, and Gauss–Newton retraining.

4.1 Overview

The search alternates an *additive* phase, which proposes ways to grow the network, and a *subtractive* phase, which proposes ways to shrink it. At every cycle a set of candidate structural edits is generated, all of them are ranked by a *cheap* first-order estimate that requires no retraining, and the best candidate is retrained and subjected to an acceptance test. When nothing helps, the topology has converged.

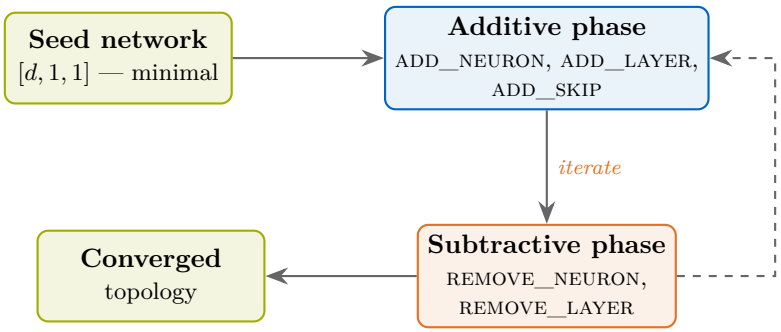


Figure 6: The ADAGOS cycle. Growth and contraction alternate, so the search can back out of a bad addition rather than being trapped by it.

Algorithm 1 ADAGOS constructive topology search

Require: seed θ, splits (train, val), thresholds $\varepsilon_a, \varepsilon_v, \tau, \rho$

repeat ▷ one cycle
  overfit guard: if depth>4 and val–train gap >20% for 3 cycles, restrict candidates to REMOVE only
  $\mathcal{C} \leftarrow \{$ADD_NEURON, ADD_LAYER, ADD_SKIP, REMOVE_NEURON, REMOVE_LAYER$\}$
  rank $\mathcal{C}$ ascending by the cheap estimate $\Delta\mathcal{L}$ ▷ no retraining, eq. (15)
  for all (edit, $\Delta\mathcal{L}$) $\in$ ranked $\mathcal{C}$ **do**
    if $\Delta\mathcal{L} > -\rho\,|J|$ **then stop** ▷ nothing helps ⇒ converged
    end if
    $\theta' \leftarrow$ GAUSSNEWTONRETRAIN(edit(θ))
    if edit is additive **then** accept iff $J(\theta') \le (1-\varepsilon_a)J$ **and** $v(\theta') \le (1-\varepsilon_v)v$
    else accept iff $v(\theta') \le (1+\tau)\,v$
    end if
    if accepted **then** $\theta \leftarrow \theta'$; **break**
    end if
  end for
  if no acceptance for `patience` cycles **then stop**
until converged
return PRUNESMALLWEIGHTS(θ)

4.2 The objective, and where parsimony comes from

The training objective is a *pure* sum of squares over the n_c layers of the network,

$$J(\Gamma, W) \;=\; \|X^{n_c} - Y\|_2^2, \tag{11}$$

with Γ denoting the topology and W the weights. There is no complexity penalty, no weight decay, no explicit $|\theta|$ term.

This deserves emphasis, because it is counter-intuitive for an algorithm whose purpose is small networks:

> **Parsimony in Adagos is structural, not a penalty.** It arises from (i) starting at a minimal topology, (ii) growing only greedily and only when growth measurably reduces error, (iii) allowing the subtractive phase to reclaim structure that stops paying, and (iv) stopping on a train/validation criterion. Adding a complexity term to eq. (11) would be a departure from the method, not an improvement to it.

The corollary, which section 5 makes vivid, is that when validation error tracks training error indefinitely — as it does on noise-free data — none of these four brakes engages, and the search grows without bound.

4.3 The Lagrangian saliency estimate

The expensive part of any topology search is evaluating candidates: naively, each requires retraining. Adagos avoids this with a first-order estimate derived from a constrained formulation of the loss in which each layer's forward relation is imposed as an explicit constraint carrying a multiplier Λ_i:

$$\mathcal{L}(\Gamma, W, X, \Lambda) \;=\; \underbrace{\|X^{n_c} - Y\|_2^2}_{J(\Gamma,W)} \;+\; \sum_{i=1}^{n_c} \Lambda_i^\top \Big(X^i - \varphi_i(W_{i-1} X^{i-1} + b_{i-1}) \Big). \tag{12}$$

Stationarity with respect to the layer states X^i yields a backward recursion for the multipliers — structurally the adjoint (backpropagation) sweep:

$$\Lambda_{n_c} = 2\,(X^{n_c} - Y)\,, \tag{13}$$

$$\Lambda_i = W_i^\top \Big(\varphi'(z^{i+1}) \odot \Lambda_{i+1} \Big), \qquad i = n_c - 1, \dots, 1. \tag{14}$$

The multiplier Λ_i measures the sensitivity of the total loss to a violation of layer i's forward relation. That is exactly what a structural edit produces: a candidate topology Γ'' leaves the frozen states $X^\star$ and multipliers $\Lambda^\star$ no longer mutually consistent, and the size of the resulting mismatch, weighted by the multipliers, estimates the change in loss without any retraining:

$$\Delta\mathcal{L}(\Gamma'') \;=\; \mathcal{L}(\Gamma'', W'', X^\star, \Lambda^\star) - J(\Gamma^\star, W^\star) \;=\; \sum_i \Lambda_i^{\star\top} \Big(X_i^\star - \varphi_i(W_{i-1}'' X_{i-1}^\star) \Big). \tag{15}$$

Candidates are then selected by

$$\Gamma^{\text{next}} \;=\; \arg\min_{\Gamma''} \; \max(\Delta\mathcal{L}(\Gamma''),\, -J), \tag{16}$$

the inner max clipping at $-J$ because no edit can reduce a sum-of-squares loss below zero.

The practical payoff is large: *all* candidate edits are ranked with one forward and one backward pass over the frozen network, and only the top-ranked candidate is actually retrained.

4.4 Gauss–Newton retraining

Once a candidate is chosen it must be retrained. Because eq. (11) is a least-squares objective, the natural optimiser is Gauss–Newton rather than a first-order method. Writing the residual and its Jacobian

$$\boldsymbol{r}(\boldsymbol{w}) = \operatorname{vec}\left(f_{\boldsymbol{w}}(X) - Y\right) \in \mathbb{R}^{mN}, \qquad \mathbf{J} = \frac{\partial \boldsymbol{r}}{\partial \boldsymbol{w}}, \tag{17}$$

the exact Hessian of $\|\boldsymbol{r}\|^2$ is $2\mathbf{J}^\top\mathbf{J} + 2\sum_k r_k \nabla^2 r_k$. Gauss–Newton drops the second term — justified when residuals are small or the model is nearly linear in $\boldsymbol{w}$ — leaving the curvature estimate $\mathbf{H} \approx \mathbf{J}^\top\mathbf{J}$, available from first derivatives alone. We use the damped Levenberg–Marquardt form,

$$(\mathbf{J}^\top\mathbf{J} + \mu\mathbf{I})\,\delta\boldsymbol{w} = \mathbf{J}^\top\boldsymbol{r}, \qquad \boldsymbol{w} \leftarrow \boldsymbol{w} - \delta\boldsymbol{w}, \qquad \mu = \max\Big(\mu_0,\ 10^{-2}\cdot\tfrac{1}{P}\operatorname{tr}(\mathbf{J}^\top\mathbf{J})\Big), \tag{18}$$

with μ increased on a rejected step and decreased on an accepted one, and the step accepted only if the loss decreases. That monotonicity guard matters: without it a single overshooting step can leave a candidate *worse* than the network it was grown from, which inverts the ranking and stalls growth entirely. Several random restarts are run and the best kept, since the loss is non-convex.

Cost note. The Gauss–Newton step is $\mathcal{O}(MP^2 + P^3)$ in the parameter count P, against $\mathcal{O}(MP)$ for a first-order method. The search is therefore cheap *only while the network stays small*. This single fact explains the cost asymmetry reported in section 7.2.

4.5 Candidate generation: every edit begins as a no-op

The estimate eq. (15) is first-order and therefore only trustworthy in a neighbourhood of the current network. A candidate initialised at random would sit far outside that trust region and its $\Delta\mathcal{L}$ would be meaningless. ADAGOS therefore initialises every structural edit to be *function-preserving*: at the moment of insertion the network computes exactly what it did before, so $\Delta\mathcal{L} \approx 0$ and the ranking measures the edit's *potential*, not the damage caused by its initialisation.

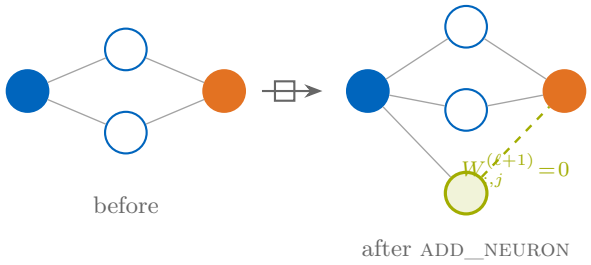


Figure 7: Function-preserving ADD_NEURON: the new unit is given a small random fan-*in* but an exactly zero fan-*out*, so it cannot yet change the output. ADD_LAYER uses the analogous device — an SVD split $\mathbf{W}^\star = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ into $\mathbf{W}_1 = \alpha\mathbf{V}^\top$, $\mathbf{W}_2 = \alpha^{-1}\mathbf{U}\mathbf{\Sigma}$ so that $\mathbf{W}_2\mathbf{W}_1 \approx \mathbf{W}^\star$ in the near-linear regime $\tanh z \approx z$; ADD_SKIP enters at zero weight.

The scale factor α in the layer split is a genuine trade-off. Taking $\alpha \to 0$ recovers the exact SVD identity but makes $\mathbf{W}_2$ enormous and $\mathbf{J}^\top\mathbf{J}$ badly conditioned, so Gauss–Newton diverges. We use $\alpha = 0.3$, which keeps the split close enough to identity for the estimate to hold while leaving the retraining problem well conditioned.

4.6 The acceptance gate

Ranking chooses *which* edit to try; the gate decides whether to keep it. The two directions are deliberately asymmetric:

- An **additive** edit must *earn* its parameters: it is accepted only if it reduces training error by at least a relative margin ε_a *and* does not worsen validation error. The margin is what prevents the network from buying vanishing improvements at fixed cost.

- A **subtractive** edit need only be harmless: it is accepted if validation error does not worsen by more than a small tolerance τ. Free removals are always taken.

Together with the overfit guard (line 2 of algorithm 1), which switches the search into removal-only mode when the validation–training gap persists, these rules constitute the entirety of the parsimony mechanism discussed in section 4.2.

4.7 Implementation notes

Several details were necessary to make the method behave as specified; we record them because they are the difference between a working and a stalling search.

- **Subtractive estimates are exact.** For REMOVE_NEURON the estimate is computed analytically on frozen weights by zeroing the dropped unit's outgoing column; REMOVE_LAYER merges $\mathbf{W}_{\text{next}}\mathbf{W}_{\text{prev}}$. No gradient steps are needed, so these candidates are essentially free.
- **Additive estimates use a short warm-up.** Because a function-preserving addition has exactly zero gradient signal at insertion, a few gradient steps are taken before evaluating eq. (15), so that the new structure has begun to express something rankable.
- **Monotone retraining.** As noted in section 4.4, the retrainer is seeded with the input network's loss and returns the input network if no step improves on it. Without this, a wider candidate could return a worse loss than a narrower one and every additive move would look harmful.
- **Search budget must be verified.** A capped cycle count silently produces a converged-looking result that is merely truncated. Every run records whether it used its full budget, and truncated runs are excluded from all reported tables.

4.8 Relation to prior art

ADAGOS sits within a lineage of constructive and structured-network patents. Table 1 locates it against the closest of them. The recurring distinction is the *selection signal*: where earlier methods grow or connect networks by heuristics (raw gradient magnitude, hand-designed skips, staged insertion), ADAGOS decides every edit — growth, contraction, and skip connection alike — by the single Lagrangian estimate of eq. (15), and retrains the winner with zero-memory Gauss–Newton under a validation-gated stop.

Table 1: ADAGOS against the nearest prior-art patents.

Patent	Covers	Relation to Adagos
US 11,521,068 (NeST)	Constructive growth + magnitude pruning	Closest structural prior art. ADAGOS differs by ranking on $\Delta\mathcal{L}$ rather than raw gradient magnitude, retraining with zero-memory Gauss–Newton, and stopping on an automatic validation gate.
US 2019/0130278 (Karras)	Progressive layer insertion (GANs)	Prior art for growing by layer insertion; ADAGOS uses the Lagrangian to decide *which* insertion is worth its parameters.
US 11,158,096 (Schubert)	Encoder–decoder with skip connections	Prior art for skips; ADAGOS *discovers* them automatically via ADD_SKIP scored by $\Delta\mathcal{L}$, rather than hand-designing.
US 2018/0322388 (Shea)	Joint encoder–decoder autoencoder training	Prior art for the compression block; the novelty is applying the topology search to it with the same estimator.
US 7,814,038 (Repici)	Modified backprop for feedback networks	Not architecture search, but relevant to how the multipliers of eq. (14) propagate correct error signals through non-standard (skip/feedback) connection paths.

5 Experiments on Synthetic Data

Before applying either method to a real motor, we characterise them on a family of problems where the ground-truth function is known exactly and where the two factors we care about — input dimension and observation noise — can be varied independently. This separation is the point of the section: on real data the two are confounded.

5.1 Datasets

Both datasets are built from the same quadratic form

$$f(\boldsymbol{x}) = (\boldsymbol{x} - \mathbf{1})^\top \mathbf{A}\,(\boldsymbol{x} - \mathbf{1}), \qquad \mathbf{A} \succ 0, \quad \boldsymbol{x} \in [-3, 3]^d, \quad N = 5000, \tag{19}$$

with $\mathbf{A}$ symmetric positive definite and $d \in \{1, 2, 3\}$.

Dataset A (clean).
$y = f(\boldsymbol{x})$ exactly. A near-zero residual is attainable, so there is *no error floor*: more capacity always buys more accuracy, without limit.

Dataset B (noisy).
$y = f(\boldsymbol{x}) + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ with σ equal to 12.5 % of the peak-to-peak range of f. This

imposes an irreducible MSE floor: past a certain capacity, additional parameters can only fit noise.

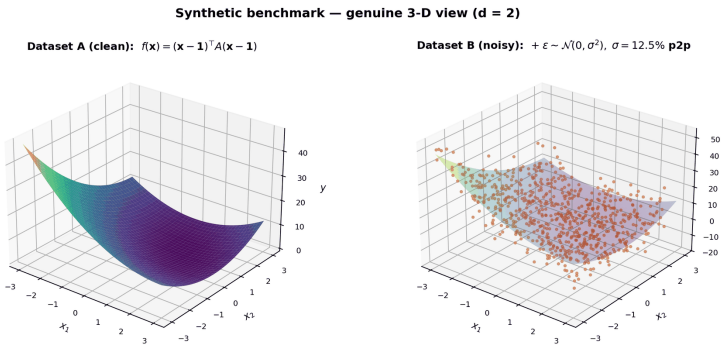


Figure 8: The clean quadratic surface for $d = 2$. Dataset B is this same surface with Gaussian noise added to every observation. The pairing is deliberate: A and B differ *only* in the noise term, so any behavioural difference between them is attributable to noise alone.

5.2 Protocol

Pruning track.
A baseline MLP is selected by Optuna TPE (≤ 240 trials), taking the *smallest* architecture whose validation MSE is within 5 % of the best observed — i.e. the baseline is already size-conscious, not merely accurate. Each pruning step is followed by the fine-tune and LBFGS polish of section 3.6. Because a single-initialisation baseline can land in a poor basin and thereby flatter every pruning method that happens to re-initialise, baselines are retrained from multiple initialisations and the best-on-validation is locked in before any pruning is run.

Constructive track.
ADAGOS from the minimal seed $[d, 1, 1]$, on the *identical* train/validation split, run to full convergence with budget to spare.

Table 2: Tuned baseline MLPs (Optuna, tanh activation). These are the models the pruning track compresses and the reference against which both tracks are scored.

Dataset	d	Architecture	Params	Test MSE
A	1	1×8	25	6.24×10^{-8}
A	2	1×16	65	2.47×10^{-5}
A	3	3×32	2273	1.27×10^{-4}
B	1	1×4	13	3.95
B	2	1×4	17	27.43
B	3	1×6	31	622.8

Note already in table 2 the asymmetry that will dominate the section: on clean data the tuner keeps buying capacity as d grows ($25 \to 65 \to 2273$ parameters), while on noisy data it settles at 13–31 parameters regardless of d.

5.3 Pruning results

Table 3: Best pruning operating point per method: **sparsity reached** within the error budget, with the resulting MSE ratio to baseline underneath. Higher sparsity at a ratio near 1.00× is better. Green marks the best result in each column.

	Dataset A (clean)			Dataset B (noisy)		
Method	d=1	d=2	d=3	d=1	d=2	d=3
OBS	10% / 1.07	20% / 0.99	30% / 2.44	**30% / 1.00**	**40% / 1.03**	50% / 1.20
OBD	**10% / 1.00**	50% / 27	10% / 1.31	30% / 1.02	**40% / 1.03**	**70% / 1.36**
Taylor-FO	10% / 1.14	**10% / 0.99**	10% / 1.60	30% / 1.02	30% / 1.32	50% / 1.00
SNIP	30% / 22	10% / 3.97	10% / 10	**50% / 1.25**	20% / 1.10	30% / 1.48
IMP+Rewind	10% / 86	10% / 5.93	10% / 16	30% / 1.25	20% / 1.10	30% / 1.66
Magnitude	20% / 150	10% / 3.50	10% / 2.30	10% / 1.01	10% / 1.30	10% / 1.39
Structured	40% / 9.97	40% / 72	20% / 3.78	20% / 2.91	10% / 1.32	10% / 1.00
Activation	20% / 8.96	40% / 94	40% / 6.21	20% / 2.89	10% / 1.32	**10% / 1.00**
Redundancy	20% / 150	20% / 123	10% / 2.81	10% / 1.01	10% / 1.30	10% / 1.18

Two patterns are immediate. First, the *loss-aware* criteria — OBS, OBD, Taylor-FO — dominate: they are the only ones that stay near 1.00× while removing a meaningful fraction of the network. This is the expected reward for using curvature: on networks this small the Hessian is cheap and exact, so the theory's advantage is fully realised.

Second, and more striking, the clean and noisy columns behave like different problems. On dataset B every method finds *some* usable operating point at 30–70% sparsity. On dataset A the ratios explode — 22×, 86×, 150× baseline error for a mere 10–30% sparsity. The clean baselines are so well optimised that removing anything at all is catastrophic.

5.4 Constructive results

Table 4: ADAGOS outcomes from the minimal seed. "Params ↓" is the parameter reduction relative to the tuned baseline of table 2 (positive = leaner). Green marks cases where the constructed network is both leaner and competitive.

DS	d	Seed	Final topology	Test MSE	Params ↓	MSE ×base
A	1	$[1,1,1]$	$[1,3,6,5,1]$	3.55×10^{-6}	−184%	56.86×
A	2	$[2,1,1]$	$[2,13,16,5,1]$	3.38×10^{-5}	−445%	1.37×
A	3	$[3,1,1]$	$[3,17,13,3,1]$	3.15×10^{-4}	+82%	**0.55×**
B	1	$[1,1,1]$	$[1,2,1]$	3.97	+46%	**1.01×**
B	2	$[2,1,1]$	$[2,3,1,1]$	27.65	+12%	**1.01×**
B	3	$[3,1,1]$	$[3,2,1]$	625.1	+65%	**1.00×**

On noisy data the result is exactly what the method promises: from a seed that required no design decisions, the search lands within 1 % of a baseline that cost up to 240 tuning trials, using *fewer* parameters in every case. On clean data the same algorithm over-grows badly at low dimension — $[1,3,6,5,1]$, 71 parameters, to solve a one-dimensional quadratic — while paradoxically *winning* at $d = 3$, where it reaches 0.55× the baseline error with 400 parameters against the baseline's 2273.

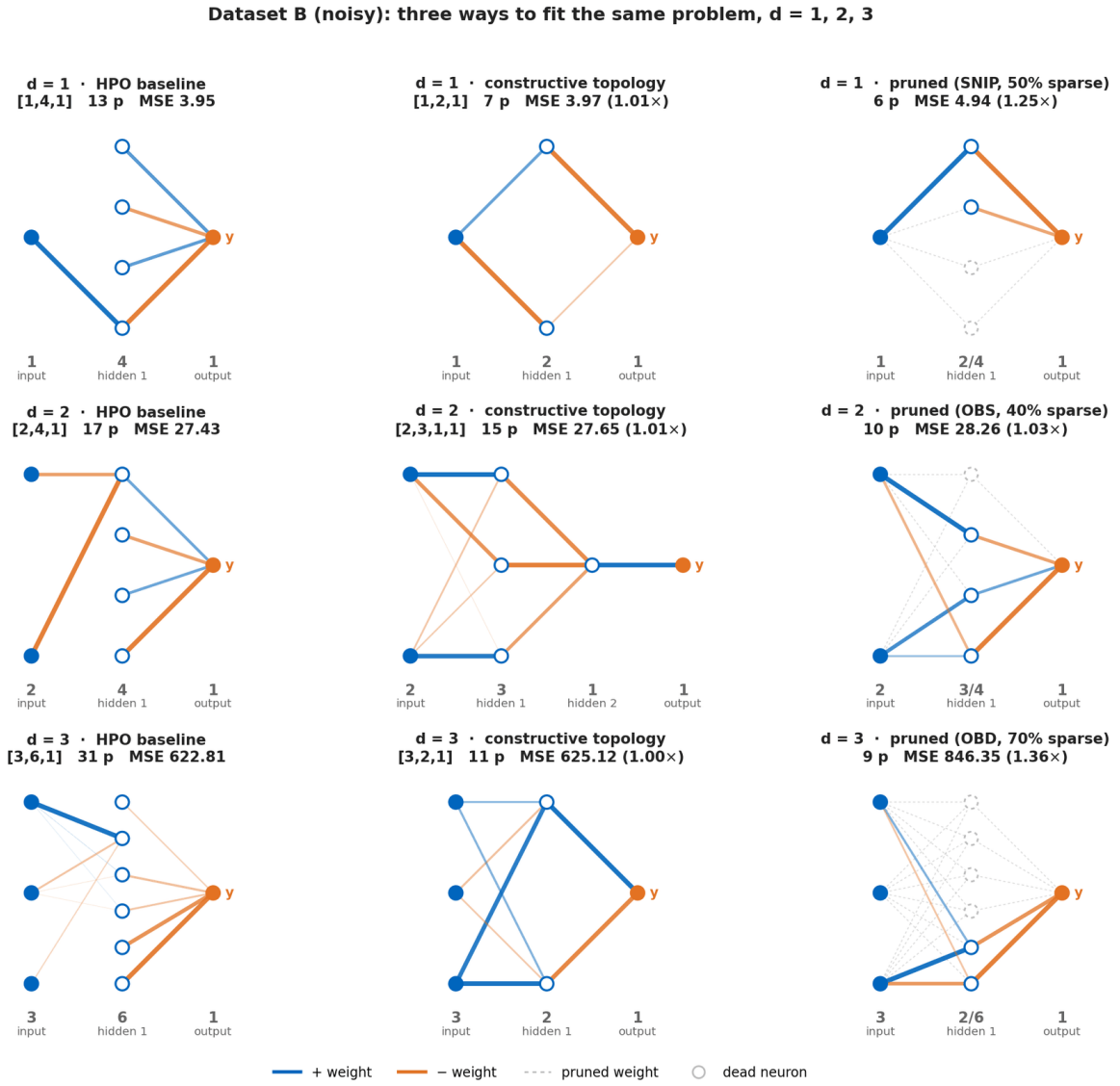


Figure 9: Dataset B, all three dimensions: tuned baseline (left), constructed network (centre), best pruned network (right). Edge width and opacity encode $|w|$, colour encodes sign. The constructed networks are visibly minimal — two or three hidden units suffice once noise sets an error floor.

5.5 Four findings

Finding 1 — Noise, not dimension, sets leanness and prunability

The clean/noisy contrast is not a detail; it is the organising variable of the entire study.

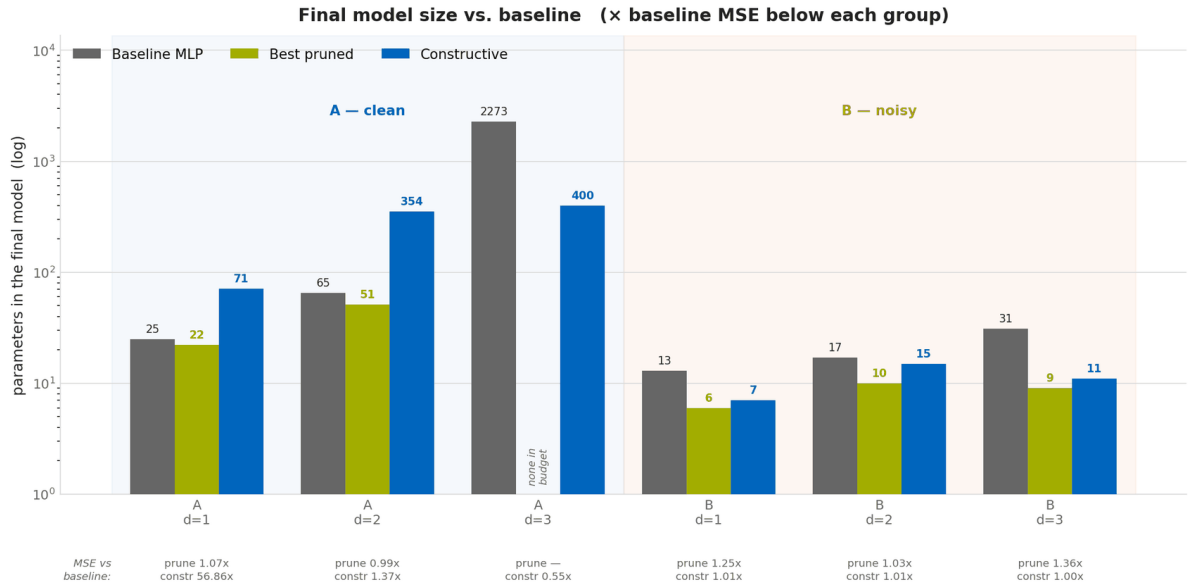


Figure 10: Left: parameters grown by the search, by input dimension. On clean data growth is unbounded ($71 \to 354 \to 400$); on noisy data it saturates at 7–15 regardless of d. Right: sparsity reachable within the error budget. The two panels are the same mechanism seen from opposite directions — noise creates an error floor, the floor creates slack, and slack is what both methods consume.

The mechanism is one sentence. *Noise creates an irreducible error floor; the floor creates slack; both compression routes spend that slack.* Where there is no floor — clean data — there is no slack: the constructive search never sees a reason to stop growing, and the pruner never finds a weight it can afford to remove. Input dimension, the variable one would intuitively expect to govern model size, is comparatively irrelevant: dataset B needs 7 parameters at $d = 1$ and 11 at $d = 3$.

Finding 2 — Without noise, the growth has no brake

On clean data ADAGOS keeps accepting additions long past the point of practical return.

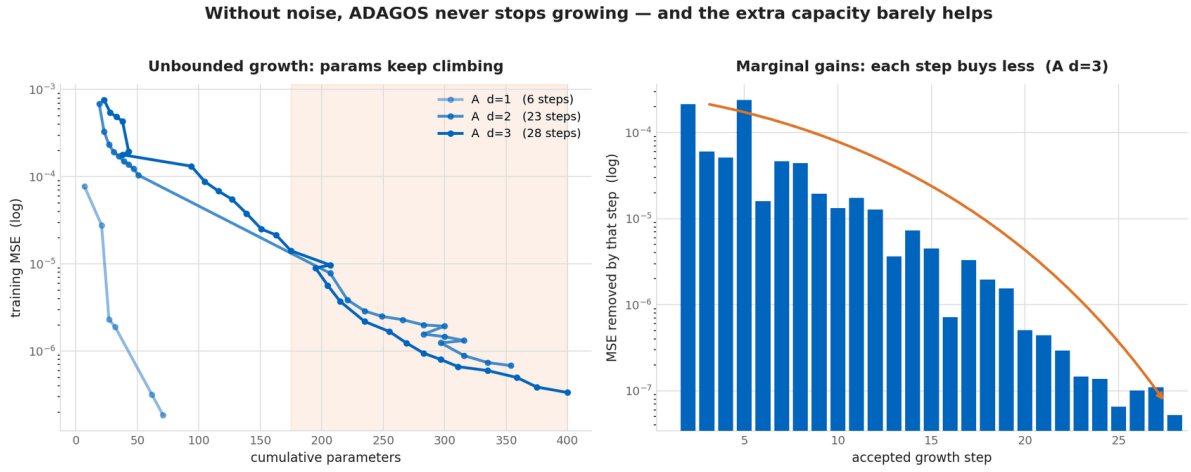


Figure 11: Left: training MSE against cumulative parameters for dataset A. Right: MSE removed by each accepted growth step, which decays by roughly three orders of magnitude across the search. Every one of these steps passed the acceptance gate: each was a genuine *relative* improvement, and the objective (eq. (11)) contains no term that would price the parameters they cost.

This is the practical consequence of section 4.2. The gate asks "did error drop by at least

ε_a in relative terms?", and on noise-free data the answer stays yes essentially forever, because validation error tracks training error down and the overfit guard never fires. The behaviour is faithful to the method as specified; it is also the clearest candidate for improvement (section 8).

Finding 3 — There is no universal winner

Neither family dominates, and the ranking of pruning criteria itself flips with problem scale.

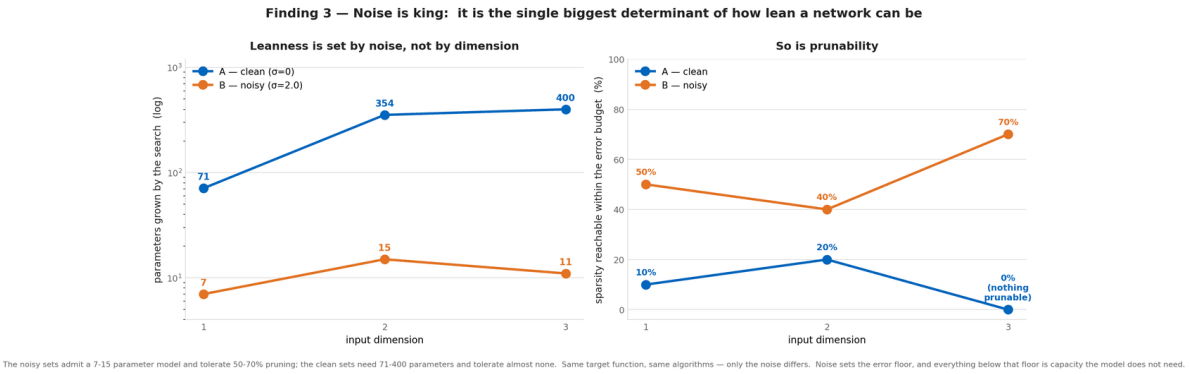


Figure 12: Final parameter counts for all three approaches across the six problems, with the MSE ratio to baseline printed beneath each group. On noisy problems the constructed and pruned networks converge to comparable size at comparable error; on clean low-dimensional problems the tuned baseline is unbeatable; on the clean high-dimensional problem the constructed network wins outright, and pruning finds nothing within budget at all.

The honest summary of fig. 12 is a partition rather than a winner:

- **Noisy problems (B, all d):** constructive and pruning tie on error (1.00–1.01×), with the constructed network reached without any architecture search.
- **Clean, low-dimensional (A, $d = 1, 2$):** the tuned baseline wins. The search over-grows and pruning has no slack to work with.
- **Clean, high-dimensional (A, $d = 3$):** the constructed network wins decisively — 0.55× the baseline error at 82 % fewer parameters — and no pruning level meets the budget.

Finding 4 — The discovered topology is sensitive to initialisation

Architecture is not a stable fingerprint of the problem; accuracy is.

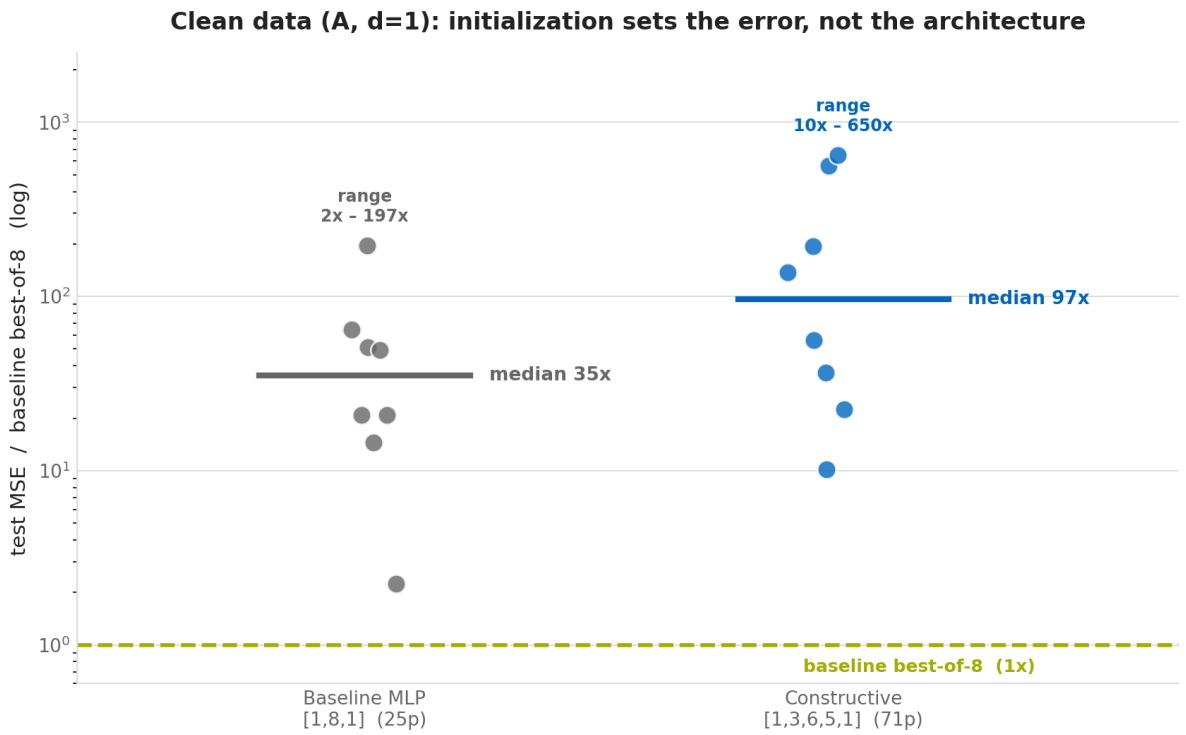


Figure 13: Eight weight-initialisation seeds on the same problem with the data split held fixed. **Top (dataset A, clean):** eight runs, eight distinct topologies, yet all reach $\approx 1 \times 10^{-7}$ test MSE. **Bottom (dataset B, noisy):** all eight runs converge to the identical $[1, 2, 1]$ network at MSE 0.17.

Figure 13 isolates weight initialisation from the data split and resolves what the topology spread actually means. On clean data the loss surface has a continuum of near-equivalent optima — many different architectures express the same quadratic to within 1×10^{-7} — so a greedy search with a local optimiser lands wherever its initialisation leads it. The function is reproducible even though the architecture is not. On noisy data, where the error floor makes the optimum genuinely discrete, all eight seeds agree exactly.

The practical reading: on the data regime the method was designed for — noisy, real-world measurements — the search is reproducible. Elsewhere, one should report the accuracy distribution over seeds and treat any single discovered topology as one sample from a set of equivalent solutions.

6 The Electric-Motor Task

The synthetic study establishes mechanism; this section establishes relevance. We apply both routes to the problem the project exists to solve: estimating PMSM stator-winding temperature from drive signals.

6.1 Dataset

We use the Kaggle *Electric Motor Temperature* benchmark [5], a test-bench dataset from a real PMSM: 1 330 000 samples at 2 Hz (≈185 h of operation) across 69 independent drive profiles. The regression target throughout is `stator_winding` temperature, ranging 18–141 °C (mean 66, sd 29 °C); the windings are the hottest, fastest-reacting and most safety-critical part of the machine, which is why they are the fixed target across every configuration below. The measured signals fall into the physically distinct groups of fig. 14.

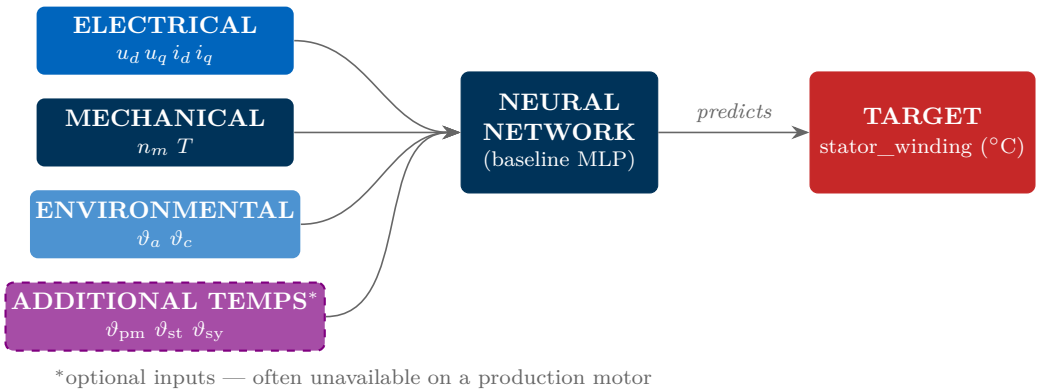


Figure 14: Signal roles in the PMSM dataset. Electrical, mechanical and environmental channels are always available; the three additional temperature channels (ϑ_{pm} magnet, ϑ_{st} stator tooth, ϑ_{sy} stator yoke) are treated as *optional*, because a deployed sensorless motor typically will not carry them.

The additional temperature channels are treated as *optional*. Three input configurations are studied — `all_in` (11 signals), `no_temps`, and `no_temps_pm` — precisely to test how much accuracy depends on sensors a production motor may not carry. This is the "fewer sensors" half of the project motivation.

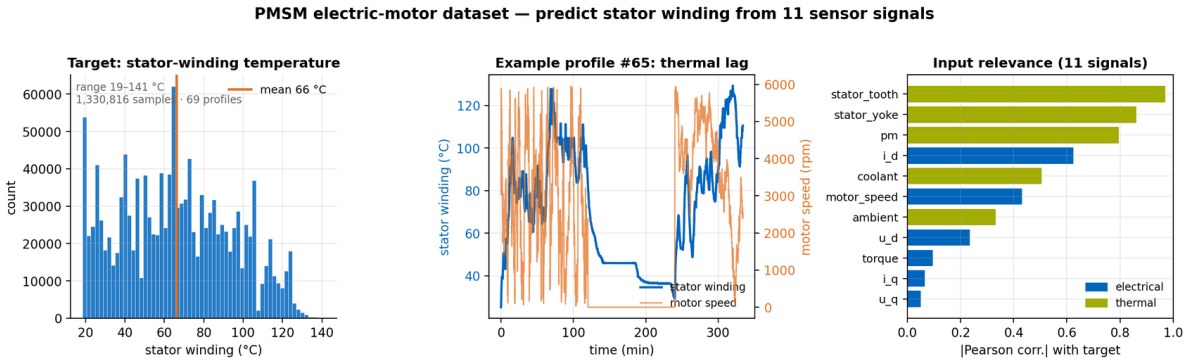


Figure 15: A representative drive session. Mechanical and electrical signals (upper two panels) are piecewise-constant test-bench set-points that switch abruptly; the temperatures they induce (lower panel) respond smoothly with a long thermal lag. That lag is the crux of the problem: winding temperature at time t depends on the *history* of the drive signals, not their instantaneous value — which motivates both the EWMA features of section 6.3 and the dynamic twin of section 6.9.

6.2 Splitting: why profiles, not samples

Consecutive samples within a drive session are separated by 0.5 s and are almost identical. A random sample-level split would therefore place near-duplicates of test points into training and report an accuracy the model cannot achieve on a genuinely unseen session. All splits in this section are made at the level of *whole profiles*.

The protocol uses one global hold-out of 14 profiles that is never seen during any search or selection, with five development splits carved from the remainder for model selection. Reported numbers are hold-out numbers.

Splitting at profile level is necessary but not sufficient: 69 sessions drawn at random can still leave whole regions of the operating envelope — a particular speed/torque combination — represented only in the test pool and never in training. We therefore build the training set by a **greedy max-diversity** selection rather than a random draw. Each session is reduced to a *fingerprint* (a resampled, z-normalised trace of speed, torque, and both currents); training then grows one session at a time, at each step adding the session *least* similar — by maximum

absolute Pearson correlation — to everything already chosen, so the training set is explicitly built to span the envelope. Validation sessions that are near-duplicates of another validation session ($|r| \geq 0.85$) are folded back into training, so the validation pool carries no redundant information either. Random and chronological split modes remain available for comparison; fig. 16 quantifies what the diversity split costs and buys.

6.3 Feature engineering

Because the target lags its drivers, instantaneous signals are an impoverished input for a memoryless model. We construct an 81-dimensional feature set:

$$\underbrace{11}_{\text{raw signals}} + \underbrace{4}_{\text{physics}} + \underbrace{(7+4)\times 6}_{\text{EWMA spans}} = 81, \tag{20}$$

where the physics terms are derived quantities (current and voltage magnitudes and power-like products) and the exponentially-weighted moving averages are taken over six time spans, giving the static model an explicit multi-scale memory of each signal's recent history. Torque is excluded from the EWMA expansion.

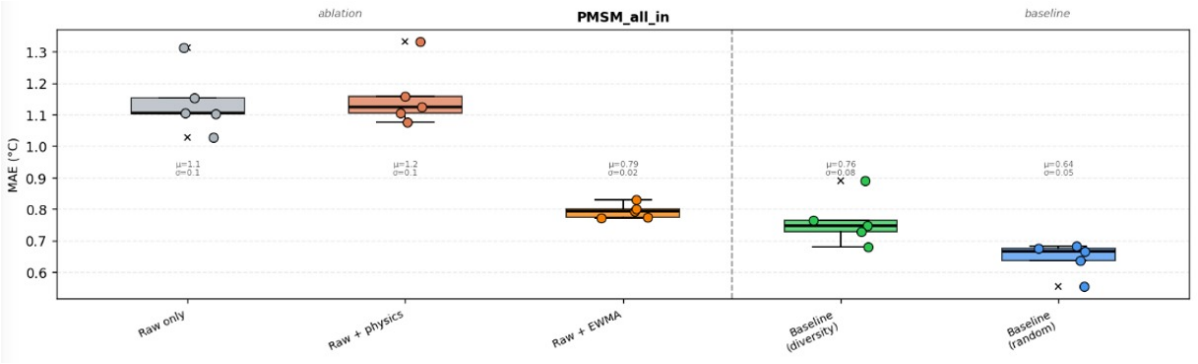


Figure 16: Ablation on the `all_in` configuration, five seeds, test MAE. *Left (feature sets):* raw signals alone and raw-plus-physics both sit near 1.1 °C; adding the EWMA history collapses the error to 0.79 °C with a much tighter spread. Temporal context, not additional instantaneous features, is what the task rewards. *Right (split discipline):* the fully-tuned baseline under the diversity split (0.76 °C) is deliberately harder than under a plain random split (0.64 °C) — a random split is more likely to place near-duplicate sessions on both sides of the boundary, the very leakage the diversity split removes, so the diversity numbers are the honest ones to carry into the compression comparison.

6.4 Hyperparameter search and final training

The baseline architecture is not hand-picked. For each input configuration an Optuna study (TPE sampler, up to 100 trials) searches depth (1–3 hidden layers), width ($\{32, 64, 128, 256\}$ units), activation ($\{\text{ReLU}, \tanh, \text{GELU}\}$), learning rate, weight decay and batch size. Every trial trains with early stopping on validation MSE (patience 50). The model finally selected is *not* the trial with the lowest validation score: following the same leanness rule used for the synthetic baselines (section 5), we take the **smallest architecture within 5 % of the best validation MSE**. This bias is deliberate — the baseline is already asked to trade a little accuracy for fewer parameters, which is exactly the trade-off the downstream compression methods exist to push further, so it should not start from an over-sized point. The selected configuration is then retrained from scratch on the full training split and evaluated exactly once on the held-out sessions; no number from that final evaluation feeds back into any tuning decision.

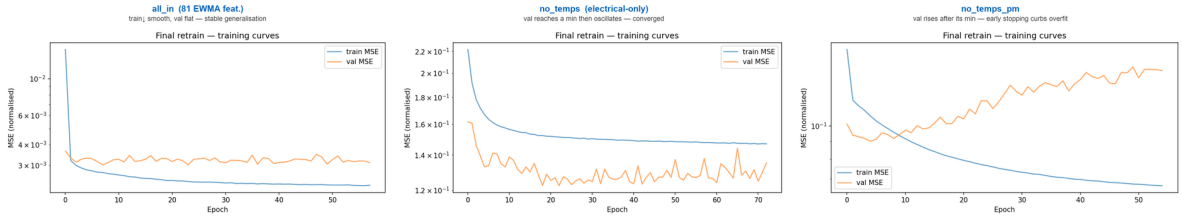


Figure 17: Final-retrain training curves for the three input configurations (train vs. validation MSE, normalised). They converge in visibly different ways: `all_in` (left) reaches a flat validation floor — stable generalisation; `no_temps` (centre) settles to a minimum and then oscillates around it — converged; `no_temps_pm` (right) shows validation rising after its minimum — textbook overfitting that early stopping truncates. These are the curves of the *selected* architecture after the Optuna search, not of the search itself.

6.5 Baseline quality

A compression study is only meaningful if the model being compressed is strong. Table 5 places our tuned baselines against published results on the same benchmark family.

Table 5: Baseline MLPs versus literature, stator-winding temperature prediction.

Model	**RMSE** (°C)	**MAE** (°C)	**Method**	**Params**
This work — `all_in` (11 in)	1.49	1.026	MLP	69 121
This work — `all_in_ewma` (81 in)	**0.805**	**0.606**	MLP	17 793
This work — `no_temps_ewma`	1.72	1.223	MLP	—
This work — `no_temps_pm_ewma`	1.18	0.754	MLP	—
Hughes et al. (2023) [4]	1.92	1.63	MLP / least squares	—
Kirchgässner et al. (2021) [6]	1.68	—	TNN	—
OLTEM (2025) [1]	1.93	—	RNN-based	—

The EWMA baseline is competitive with or better than published results, confirming that what follows is compression of a genuinely strong model rather than an easy win over a weak one. Note also the `no_temps_pm_ewma` row: at 0.754 °C MAE *without* the magnet-temperature sensor, the sensor-reduction goal is substantially achievable.

6.6 Static estimator: pruning

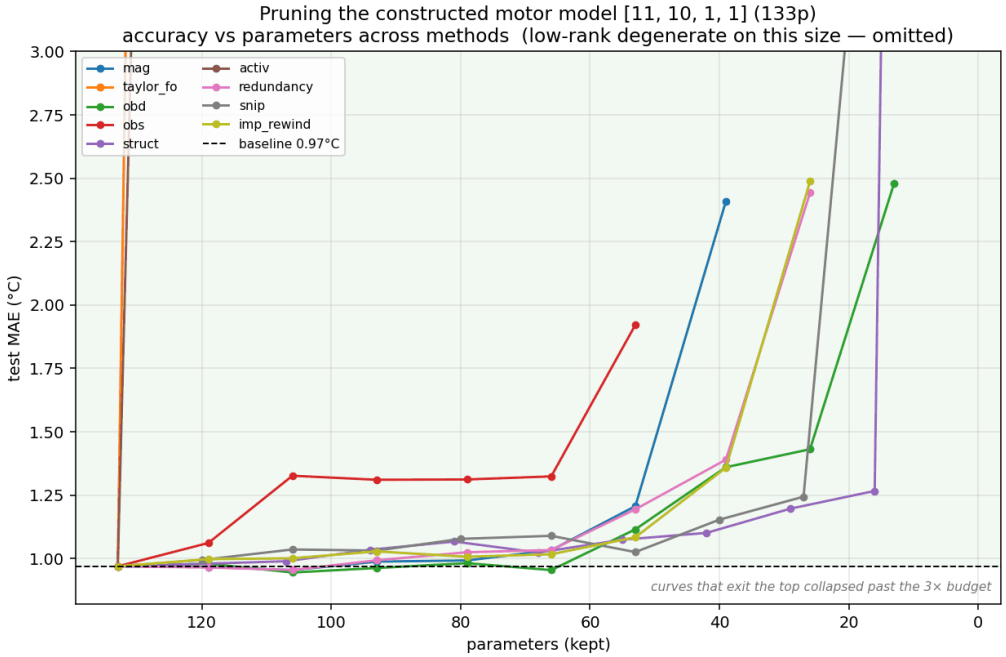


Figure 18: MSE ratio to baseline against sparsity, 11-input configuration. Most criteria track the baseline closely to 60 % sparsity before degrading. SNIP diverges catastrophically at high sparsity (3.1× at 80 %, 334× at 90 %) and is clipped here.

Table 6: Pruning on the 11-input PMSM baseline: MSE ratio to baseline by sparsity. Bold marks each method's best level; green is below baseline error.

Method	10%	20%	30%	40%	50%	60%	70%	80%	90%	**Best params**
— Baseline —	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	76k
OBD	1.012	1.032	**1.012**	1.048	1.099	1.038	1.060	1.070	1.059	53.5k
OBS	1.012	1.032	**1.012**	1.048	1.099	1.038	1.060	1.070	1.059	53.5k
Magnitude	**1.012**	1.033	1.013	1.044	1.049	1.057	1.051	1.093	1.088	68.8k
Activation	1.012	1.036	**1.010**	1.047	1.085	1.038	1.054	1.090	1.137	53.3k
Structured	1.012	1.031	**1.011**	1.047	1.084	1.055	1.034	1.031	1.059	53.3k
Redundancy	**0.973**	1.046	**0.957**	1.031	1.046	1.042	1.087	1.033	1.091	53.5k
Taylor-FO	1.012	1.035	**1.010**	1.046	1.084	1.037	1.054	1.089	1.138	53.5k
SNIP	1.197	1.160	**1.092**	1.169	1.115	1.073	1.270	3.12	333.8	30.6k
IMP+Rewind	1.291	1.278	1.253	1.272	1.240	1.186	1.113	1.115	**1.096**	**7.6k**
Low-Rank	1.000	—	—	—	—	—	—	—	—	148.2k

Compared with the synthetic results the ranking has *inverted*. The loss-aware criteria that dominated the small networks are now unremarkable, while structured pruning and IMP+Rewind — mediocre on the synthetic problems — give the best size reductions here. IMP+Rewind reaches 7.6k parameters, an order of magnitude below every other criterion, at a modest 9.6 % error cost.

6.6.1 Why OBS collapses onto OBD

In table 6 the OBS and OBD rows are *bit-for-bit* identical. This is not a coincidence but a scale limit. OBS requires $\mathbf{H}^{-1}$, an $n \times n$ matrix in the parameter count: at $n = 17{,}793$ that

is a 17,793 × 17,793 dense matrix, roughly 1.2 GB, beyond the exact-inversion threshold in the implementation. The solver falls back to the OBD diagonal saliency, and the two methods become the same algorithm.

The synthetic results bracket the transition precisely: OBS works on dataset A (321 parameters), becomes unstable on A_2d (2,241 parameters) above 10 % sparsity, and fails on A_3d (4,945 parameters).

The theoretically best criterion is the one that does not scale. OBS is the strongest saliency measure on paper and remains so in practice up to a few thousand parameters. Beyond that it is unusable without approximation (diagonal Fisher, K-FAC, or a sketched inverse). On networks of production size, treat OBS = OBD unless one of those approximations is implemented.

A second scale-related observation concerns seed sensitivity. Every criterion except SNIP and IMP+Rewind starts from the same trained baseline and is fully deterministic. SNIP and IMP+Rewind depend on a random initialisation — SNIP scores gradients at $t = 0$, IMP rewinds survivors to their initial values — so they are the only two that can vary. Across five seeds the standard deviation of their best MSE ratio is below 0.005: seed-dependent in mechanism, robust in outcome.

6.6.2 The engineered baselines compress; the raw one does not

The 11-input frontier above is the *hard* case. Running the same ten-method suite on the two EWMA-feature baselines tells the opposite story (table 7). On both EWMA configurations *every* criterion drops below the baseline error at some sparsity: the Optuna-selected EWMA baselines carry genuine, recoverable redundancy. The electrical-only `no_temps_ewma` baseline is the most compressible of all — Redundancy reaches 0.792× at 90 % sparsity, a 21 % *improvement* on the dense model — while the raw 11-input baseline, well fitted to its harder task, is barely beaten by any method.

Table 7: Best MSE ratio to baseline reached by each criterion (over all sparsity levels), for the three input configurations. Green is below baseline error; the final column names the best method overall. Both EWMA baselines compress below 1.0 under every criterion; the raw 11-input baseline does not.

Configuration	Params	OBD	Mag.	Struct.	Redund.	SNIP	Best
`all_in_ewma` (81)	17 793	**0.927**	**0.913**	**0.892**	**0.951**	**0.884**	**0.884** (SNIP)
`no_temps_ewma` (elec.)	9281	**0.829**	**0.826**	**0.851**	**0.792**	**0.862**	**0.792** (Redund.)
`all_in` (11 raw)	69 121	1.012	1.012	1.011	**0.957**	1.073	**0.957** (Redund.)

Two readings follow. First, **Redundancy is the most versatile criterion** — the only one to beat the baseline on all three configurations — because its weight-transfer step redistributes a removed unit's contribution rather than simply zeroing it. Second, compressibility is a property of the *baseline*, not the pruner: a model tuned onto rich, time-aware features has slack to reclaim, whereas one tuned tightly onto eleven raw signals does not, and no criterion manufactures redundancy that the training did not leave behind.

6.7 Static estimator: construction

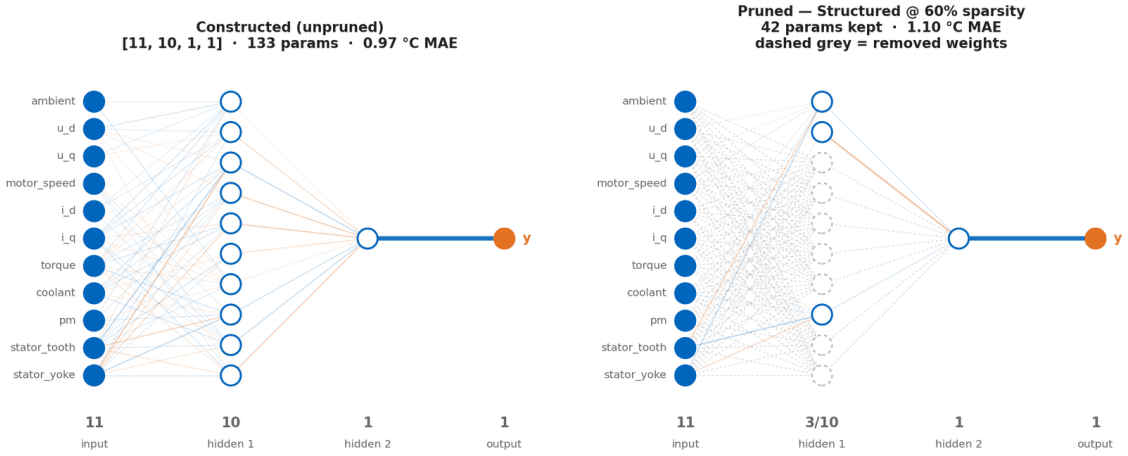


Figure 19: The constructed 11-input estimator before (left) and after (right) structured pruning at 60 % sparsity. Dashed grey edges are those pruning removed. The search selects a $[11, 10, 1, 1]$ topology — a wide first hidden layer feeding a one-unit bottleneck — and pruning then removes 60 % of its weights for 0.13 °C of additional error.

Table 8: Constructed estimators and their pruned variants, hold-out MAE.

Model	**Topology**	**Params**	**Sparsity**	**Test MAE** (°C)
Constructed winner (11-in)	$[11, 10, 1, 1]$	133	0%	0.97
↪ best pruned — Structured	$[11, 10, 1, 1]$	**42**	60%	1.10
Constructed winner (81-in)	$[81, 3, 1]$	250	0%	0.92
↪ best pruned — IMP+Rewind	$[81, 3, 1]$	**75**	70%	**0.79**

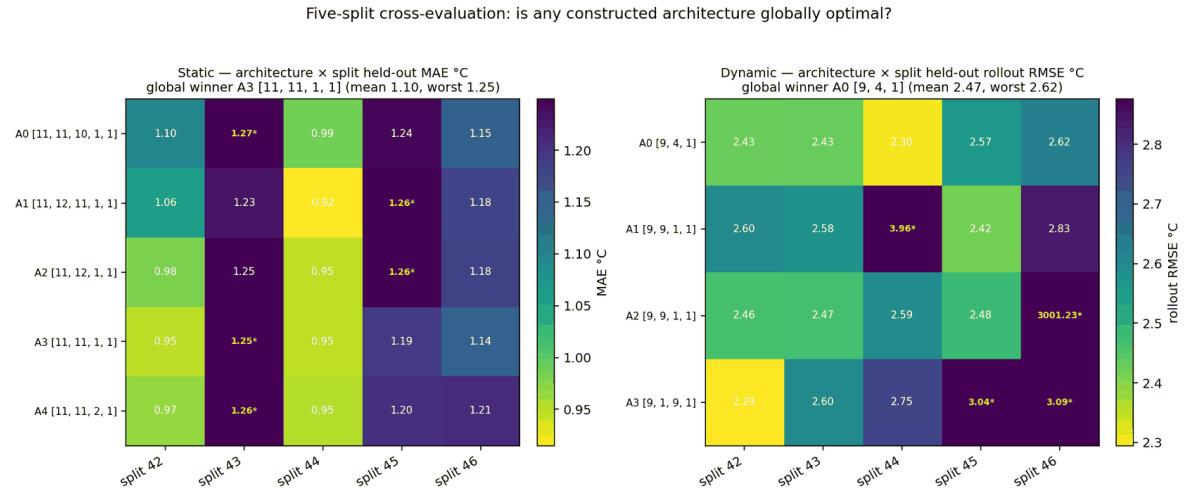


Figure 20: Five-split cross-evaluation of candidate constructed topologies, to test whether any single architecture is globally best. *Left (static):* held-out MAE for five near-winner topologies across five profile splits — no topology wins every split, but all cluster within $1.1 \pm 0.1\,°C$, so the choice of a compact representative is not critical. *Right (dynamic twin):* the analogous free-running rollout-RMSE matrix; most cells sit near $2.5\,°C$, but isolated topology×split combinations diverge outright (e.g. the $3001\,°C$ cell), which is exactly why the dynamic operating point is chosen on rollout stability rather than one-step error (section 6.9).

6.8 Head-to-head

Table 9: All four routes on the PMSM task. ADAGOS beats the tuned baseline on 11 inputs, trails it on 81, and is dramatically leaner on both.

Model	**Topology / architecture**	**Params**	**Sparsity**	**Test MAE** (°C)
Baseline MLP (11-in)	2L · 256n · ReLU	69 121	0%	1.03
↪ pruned · Redundancy	≈30% sparse	≈48 400	30%	0.98
Constructed · ADAGOS (11-in)	[11, 10, 1, 1]	**133**	0%	**0.97**
↪ pruned · Structured	[11, 10, 1, 1]	**42**	60%	1.10
Baseline MLP (81-in)	4L · 64n · GELU	17 793	0%	0.61
↪ pruned · SNIP	≈60% sparse	≈7100	60%	**0.54**
Constructed · ADAGOS (81-in)	[81, 3, 1]	**250**	0%	0.92
↪ pruned · IMP+Rewind	[81, 3, 1]	**75**	70%	0.79

The 11-input column is the strongest result in this report. ADAGOS reaches $0.97\,°C$ MAE with **133 parameters** against the tuned baseline's $1.03\,°C$ with **69,121** — better accuracy at 520× fewer parameters, and reached without any architecture or hyper-parameter search.

The 81-input column is the honest counterweight. Given rich engineered features the tuned baseline wins on accuracy ($0.61\,°C$ against $0.92\,°C$), and the search settles for a shallow $[81, 3, 1]$ network: it grows too *little*, not too much. We attribute this to the combination of tanh saturation on an 81-dimensional input (section 2.3) and a greedy gate with no incentive to go deeper. Note, however, that construct-then-prune closes most of the gap ($0.79\,°C$ at 75 parameters), and does so at 237× fewer parameters than the pruned baseline.

6.9 Dynamic thermal twin

The static estimator is memoryless: it maps instantaneous signals to a temperature. A *dynamic twin* instead learns the temperature *increment* over one time step,

$$x(t+\Delta) \;=\; x(t) \;+\; h\big(x(t), \boldsymbol{u}(t)\big), \tag{21}$$

with x the winding temperature and $\boldsymbol{u}$ the drive signals. Because it is integrated forward from its own predictions, it can run *free* — given an initial temperature and a drive profile it produces the whole trajectory, with no temperature sensor in the loop at all.

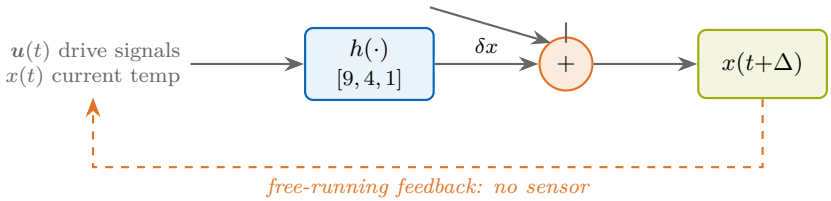


Figure 21: The dynamic twin predicts an increment which is integrated forward. In free-running mode its own output becomes the next input, so errors *compound* — a property that changes what "a good pruned model" means.

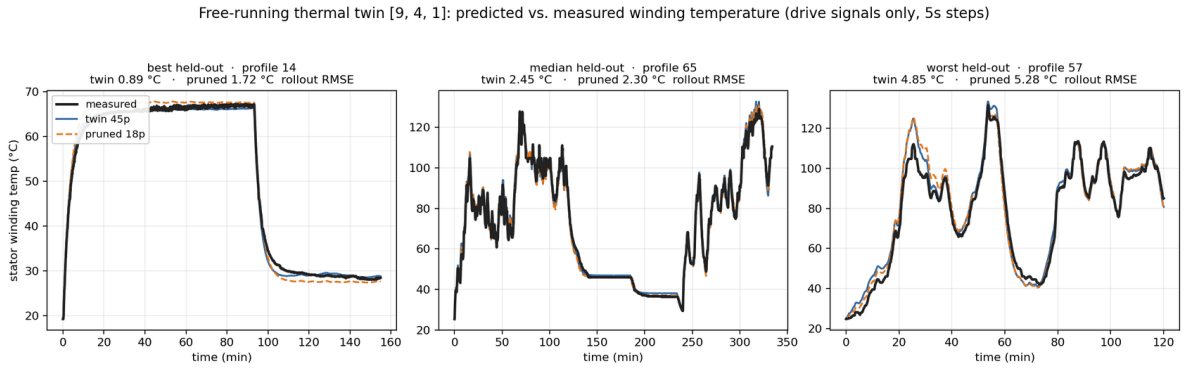


Figure 22: Free-running twin $[9, 4, 1]$ against measured winding temperature on held-out profiles, using drive signals only and 5 s steps. Best, median and worst hold-out profiles shown. The twin tracks a 100 °C excursion over several hours from an initial condition alone, with a median rollout RMSE of 2.45 °C.

6.9.1 Prune on rollout, not on one-step error

The twin exposes a pitfall with real deployment consequences. Pruning criteria score a model by its *one-step* error, because that is the loss it was trained on. For a free-running model that metric is misleading.

Table 10: Magnitude pruning of the $[9, 4, 1]$ twin: one-step error against free-running rollout RMSE. The one-step column degrades gently and monotonically; the rollout column falls off a cliff.

Sparsity	Params	One-step MSE	Rollout RMSE (°C)	Status
0%	45	1.109×10^{-4}	2.43	stable
10%	40	1.114×10^{-4}	2.49	stable
20%	35	1.239×10^{-4}	3.49	stable
30%	31	2.331×10^{-4}	19.80	degraded
40%	27	3.293×10^{-4}	103.45	diverged
50%	22	3.730×10^{-4}	89.64	diverged

At 40 % sparsity the one-step MSE has grown by a factor of only 2.97 — an entirely benign-looking number that would pass most error budgets. The same model, run free, diverges to 103 °C RMSE. A model selected on one-step error would be shipped and would fail in service.

Select on the metric the deployed system experiences. For a free-running twin that means rollout stability over the full horizon, evaluated per profile, with divergent configurations excluded outright — not the one-step loss the model was fitted on.

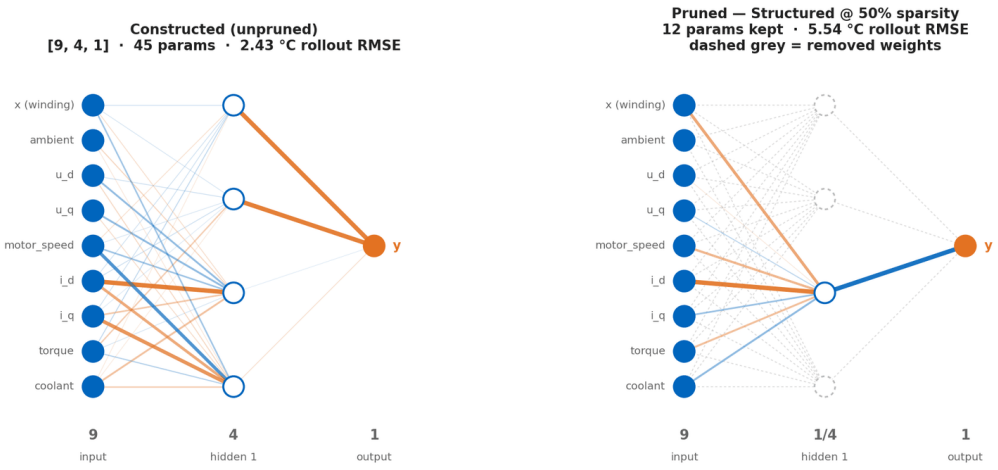


Figure 23: The constructed twin $[9, 4, 1]$, 45 parameters (left), and its structured-pruned variant at 50 % sparsity, 12 parameters (right). Structured pruning collapses the hidden layer to a single surviving unit, which retains all nine of its incoming connections and its path to the output; the three eliminated units are drawn hollow with their removed weights dashed. Edge width encodes $|w|$ within each layer. The surviving unit is the one with the richest input representation — the largest incoming-weight norm — not the largest output weight; structured pruning scores neurons on outgoing magnitude but re-polishes the network after each removal, so that unit's output weight is *re-learned* to carry the whole layer. This is why it appears faint on the left (its pre-pruning output weight is small) yet strong on the right (re-learned to ≈ 0.75). The surviving twin costs 12 parameters and reaches 5.54 °C rollout RMSE — a real but heavy degradation from the unpruned 2.43 °C, which is why the recommended dynamic operating point is a milder OBD pruning at 60 % (18 parameters, 2.59 °C).

The best dynamic operating point overall is OBD at 60 % sparsity: **18 parameters** at 2.59 °C rollout RMSE, against the unpruned twin's 45 parameters at 2.43 °C. An eighteen-parameter model that predicts a multi-hour thermal trajectory to within 2.6 °C using no temperature sensor is, by some margin, the most compelling answer this project has to its title question.

7 Evaluation

This section steps back from individual experiments and evaluates the two families against each other along the axes that matter for a deployment decision: accuracy per parameter, cost of *producing* the model, robustness, and applicability.

7.1 Accuracy per parameter

Table 11: Summary across all tasks: the leanest model that stays within the error budget, and how it compares with the tuned baseline it is measured against.

Task	Best lean model	Params	vs. baseline params	Error
Synthetic B, d=1	constructed $[1, 2, 1]$	7	0.54×	1.01× base
Synthetic B, d=2	constructed $[2, 3, 1, 1]$	15	0.88×	1.01× base
Synthetic B, d=3	constructed $[3, 2, 1]$	11	0.35×	1.00× base
Synthetic A, d=3	constructed $[3, 17, 13, 3, 1]$	400	0.18×	0.55× base
PMSM 11-in (static)	constructed $[11, 10, 1, 1]$	133	0.0019×	0.97 °C
PMSM 11-in, pruned	+ structured 60%	42	0.0006×	1.10 °C
PMSM 81-in, pruned	constructed + IMP 70%	75	0.0042×	0.79 °C
PMSM dynamic twin	constructed + OBD 60%	**18**	—	2.59 °C rollout

The headline is consistent across every real-data row: **tens to hundreds of parameters suffice for approximately** 1 °C **accuracy**, where a tuned baseline uses tens of thousands. The compression factor on the motor task is 50–500×, and the resulting models are small enough to be tabulated by hand.

7.2 Cost of producing the model

Parameter count measures the cost of *running* a model. The cost of *finding* it is a separate quantity, and it is the one that decides whether an automated method is worth adopting. We instrument both pipelines end-to-end: the baseline's cost includes its full HPO campaign (120–201 trials), and the constructive cost includes the entire search.

Table 12: Training-FLOP ratio, constructive search versus tuned-baseline pipeline. Values below 1 mean the constructive route is cheaper.

	d=1	d=2	d=3
Dataset B (noisy)	12× **cheaper**	33× **cheaper**	146× **cheaper**
Dataset A (clean)	21× dearer	624× dearer	40× dearer

The sign flips cleanly, 6 cases out of 6, and the mechanism is the one identified in section 4.4: the Gauss–Newton step costs $\mathcal{O}(MP^2 + P^3)$ against Adam's $\mathcal{O}(MP)$. The search is cheap only while the network stays small — and the network stays small only when noise stops the growth.

"No universal winner" and "noise is king" are the same statement. The variable that decides whether the constructive route is leaner (fig. 10) is the same variable that decides whether it is cheaper (table 12): the presence of an error floor. This unifies the report's two main findings under one mechanism.

A caveat on interpretation: the baseline's FLOP count is set largely by its epoch cap, which nearly all HPO trials reached. A larger training budget would widen dataset B's advantage and narrow dataset A's disadvantage — it changes the magnitudes but flips no signs.

7.3 Robustness

Table 13: Robustness of the constructive search across eight seeds. Topology varies; accuracy does not.

Setting	Topology spread	Accuracy spread
Synthetic A (clean)	8/8 distinct	all $\approx 1 \times 10^{-7}$ MSE
Synthetic B (noisy)	**8/8 identical** $[1, 2, 1]$	all 0.17 MSE
PMSM `all_signals`	8/8 distinct, 107–301 params	$1.13 \pm 0.12\,^{\circ}\mathrm{C}$
PMSM `coolant+elec`	6/8 distinct, 51–155 params	$8.76 \pm 0.59\,^{\circ}\mathrm{C}$

The pattern established synthetically in fig. 13 reproduces on real data: the discovered *architecture* wanders while the resulting *accuracy* is tight (±1 % relative on the observable configuration). The practical consequences are worth stating plainly:

- A discovered topology is **not a reliable fingerprint** of the problem. Do not read structural meaning into one run's layer widths.
- The method is nonetheless **reliable as a producer of accurate compact models**, which is what it is being asked to do.
- Reporting should therefore be distributional — accuracy mean and spread over seeds — rather than a single architecture.

The harder `coolant+elec` configuration shows a wider *relative* spread (±7 %). With no temperature sensors and only 14 hold-out profiles, this is genuine data-limited variance rather than an artefact of the search.

7.4 What worked

Competitive with no search at all.
On the raw-signal motor task ADAGOS matches and beats a fully tuned baseline while requiring zero architecture or hyper-parameter decisions. Against a baseline that consumed 121–240 HPO trials and three explicit design choices, the search consumed one automated run and none.

Construct-then-prune is the strongest recipe.
The two families are complements, not substitutes. Construction finds a good small topology; pruning then removes 60 %–70 % of what remains. The best 81-input result (0.79 °C, 75 parameters) and the best dynamic result (2.59 °C, 18 parameters) are both construct-then-prune.

Leanness with stable accuracy.
50–500× fewer parameters than the tuned baseline, with accuracy stable across data splits and seeds.

Temporal features carry the task.
EWMA history reduces MAE from 1.1 °C to 0.79 °C (fig. 16) — a larger effect than any architectural choice in this report.

7.5 What did not work, and why

Over-growth on clean data.
With no error floor, no acceptance-gate brake engages and the search grows indefinitely (fig. 11). The behaviour is faithful to the specified method — the objective is pure sum-of-squares by design (section 4.2) — which makes it a *design limitation* rather than an implementation defect. Noise-free regression is simply not the regime the method targets.

Under-growth on rich features.
The mirror-image failure. On the 81-input task the search stops at $[81, 3, 1]$ and trails the baseline. Two contributing causes: tanh saturates on high-dimensional inputs, limiting what a single added unit can contribute; and the greedy gate offers no incentive to grow *deeper* when a shallow addition yields a marginally better immediate improvement.

Not every pruner scales.
OBS reverts to OBD above a few thousand parameters (section 6.6.1); SNIP is unusable above 70 % sparsity, reaching 334× baseline error at 90 %; low-rank factorisation rarely finds a useful operating point on narrow networks.

One-step metrics mislead for dynamic models.
Table 10 is the cautionary result of the report: a 2.97× one-step degradation concealed a 42× rollout divergence.

7.6 A deployment recipe

Recommended procedure, given a new regression task.

1. **Check for an error floor.** Estimate the noise level. If the target is essentially noise-free, expect the constructive search to over-grow and budget for a tuned baseline instead.
2. **Invest in temporal/derived features first.** On the motor task this mattered more than any architectural decision.
3. **Construct from a minimal seed** rather than searching architectures — on noisy data this is both leaner and cheaper.
4. **Then prune the constructed network**, sweeping sparsity as a curve rather than picking one level. Use loss-aware criteria (OBS/OBD/ Taylor-FO) below a few thousand parameters and structural or IMP+Rewind criteria above.
5. **Select on the deployed metric** — rollout stability for a free-running model — and on a hold-out split that selection never touched.
6. **Report accuracy over seeds**, not a single architecture.

8 Next Steps

The results suggest a research agenda in three tiers: targeted fixes to the constructive algorithm's two failure modes, extensions to the experimental programme, and steps required before any of this reaches a vehicle.

8.1 Improving the constructive algorithm

A parsimony term in the acceptance gate. The over-growth of fig. 11 has a clear cause: the gate asks only whether error fell, never whether the parameters were worth it. A minimal change is to require additive edits to improve a *penalised* criterion,

$$J_\lambda(\theta) \;=\; J(\theta) \;+\; \lambda\, P(\theta), \tag{22}$$

or equivalently to require an improvement per added parameter above a threshold. This must be done carefully and reported explicitly: the patent's objective is pure sum-of-squares by design, so a penalised gate is a deliberate *departure* from the specified method, evaluated as a variant rather than presented as the method itself. The experiment is cheap — rerun the six synthetic problems over a λ sweep and locate the value at which dataset A's growth halts without harming dataset B's converged topologies.

Non-saturating activations for high-dimensional inputs. The under-growth on 81 inputs is plausibly a tanh saturation effect. Two tests would separate the hypotheses: (i) rerun the 81-input search with a non-saturating activation (GELU or Swish, both used by the winning baselines); (ii) rerun it with input normalisation adjusted so that pre-activations start well inside the linear regime. If (ii) alone closes the gap, the problem is conditioning rather than the activation function. This experiment is higher-value than it looks: the 81-input configuration is the one where the baseline currently wins, so it is where the method has most to gain.

Depth-aware candidate ranking. The greedy gate accepts whichever edit yields the best immediate improvement, which systematically favours widening over deepening — a shallow addition almost always shows a better first-order $\Delta\mathcal{L}$. Ranking candidates by improvement *per parameter added*, or allowing a short lookahead over two edits, would let genuinely useful depth compete.

Scalable second-order pruning. OBS is the strongest criterion where it is affordable and unavailable where it matters most (section 6.6.1). Implementing a diagonal-Fisher, K-FAC, or sketched inverse-Hessian approximation would test directly whether OBS's theoretical advantage survives to production scale, or whether the diagonal approximation is all that is recoverable.

8.2 Extending the experimental programme

Fold construct-then-prune into one pipeline. Currently the two stages are run and evaluated separately. Since the best results in this report come from their composition, they should become a single automated procedure with one hold-out protocol and one selection rule, so that "construct, then prune" is a method rather than a manual sequence.

Sensor-reduction study as a first-class result. The `no_temps_pm_ewma` baseline reaches 0.754 °C MAE without the magnet temperature sensor. This directly addresses the project's "fewer sensors" motivation and deserves the full treatment applied to `all_in`: constructive search, ten-method pruning sweep, and a dynamic twin. The accuracy-per-sensor frontier — how much error each removed sensor costs — is the curve an OEM would actually want.

Dynamic twin: sample interval and horizon. Rollout RMSE is U-shaped in the integration step Δ, with an optimum near $\Delta \approx 2.5\,\mathrm{s}$: smaller steps accumulate error over more integrations, larger steps make each step harder. The discovered block size grows with Δ. This trade-off should be characterised jointly with the pruning sweep, since the correct operating point is $(\Delta, \text{sparsity})$ as a pair rather than either alone.

Broaden beyond one dataset. All real-data conclusions rest on one benchmark from one test bench. Before any of these claims are treated as general, they should be reproduced on at least one further thermal dataset — ideally Schaeffler in-house measurements from a different motor design, which would also test transfer across machines.

8.3 Toward deployment

Quantisation and integer inference. Everything here counts *parameters*. On a microcontroller the currency is bytes and cycles. An 18-parameter twin quantised to int8 is a fundamentally different proposition from the same twin in float32, and quantisation interacts with pruning — the two compress along different axes and may or may not compose. Measuring actual flash footprint, latency, and energy per inference on target hardware would convert the parameter counts in table 11 into the units the cost case is actually made in.

Safety envelope and failure behaviour. A thermal estimator in a safety-critical drivetrain needs a characterised failure mode, not just a mean error. Required: worst-case error over all hold-out profiles rather than the mean; behaviour under out-of-distribution drive cycles; and a monitor that detects when the free-running twin has drifted. Table 10 shows divergence is possible and abrupt, so it must be detectable at runtime.

Hybrid physics-informed structure. Winding temperature obeys a lumped-parameter thermal network with known structure. Seeding the constructive search with that structure — or constraining discovered topologies to respect it — could combine the interpretability of a physical model with the accuracy of the learned one, and would likely improve extrapolation to unseen operating regimes, which is where a purely data-driven estimator is weakest.

8.4 Priority

Table 14: Suggested prioritisation, balancing expected value against effort.

Priority	Item	Effort	Expected value
1	Parsimony term in the gate	low	fixes the clearest known limitation
2	Non-saturating activation, 81-in	low	targets the one task where we lose
3	Unified construct-then-prune pipeline	medium	makes the best recipe reproducible
4	Sensor-reduction frontier	medium	directly serves the industrial goal
5	Quantisation & on-target measurement	medium	converts parameters into cost
6	Scalable OBS approximation	high	resolves an open methodological question
7	Second dataset / transfer study	high	required for generality claims

9 Conclusion

We set out to determine how lean a neural network can be while still solving a real electric-motor thermal-estimation task, and which of the two opposing compression philosophies finds that network most reliably.

How lean. Very. On the raw-signal PMSM task a constructed network of **133 parameters** reaches 0.97 °C MAE where a fully tuned baseline needs **69,121** to reach 1.03 °C — better accuracy at 520× fewer parameters. Pruning the constructed network takes it to 42 parameters at 1.10 °C. On the feature-rich variant, construct-then-prune gives 0.79 °C at 75 parameters. Most strikingly, an **18-parameter** dynamic twin predicts multi-hour winding-temperature trajectories to 2.59 °C rollout RMSE using no temperature sensor at all.

Which route. Neither dominates, and the variable that decides is not the one intuition suggests. It is not input dimension, model size, or task difficulty — it is **observation noise**. Noise creates an irreducible error floor; the floor creates slack; and both compression routes

exist to spend that slack. Where noise is present the constructive search converges to a minimal network, does so reproducibly across seeds, and costs 12–146× fewer training FLOPs than the baseline's HPO campaign. Where it is absent the search grows without brake and the tuned baseline wins. The same mechanism governs prunability: the noisy problems admit 50 %–70 % sparsity, the clean ones almost none.

Practical recipe. The two families turned out to be complements rather than rivals. Construction supplies a good small topology without any architecture search; pruning then removes a further 60 %–70 % of it. Every best result in this report is a composition of the two. Two selection disciplines proved essential: choose the operating point on a hold-out split that selection never touched, and score a free-running model on rollout stability rather than the one-step loss it was trained on — a distinction that separated a model which looked 2.97× worse from one that in fact diverged by a factor of forty.

Outlook. The clearest opening is the constructive method's missing parameter cost: a parsimony term in the acceptance gate would address the one failure mode we can name precisely, and a non-saturating activation would target the one task on which the method currently loses. Beyond that, the path to a vehicle runs through quantisation, on-target measurement, and a characterised failure envelope.

The question in the title admits a concrete answer. For this class of industrial regression problem, **tens to hundreds of parameters are enough** — one to three orders of magnitude below what a conventionally tuned network uses — and finding them does not require the tuned network first.

References

[1] Oltem: Lumped thermal and deep neural model for pmsm temperature estimation. *AI (MDPI)*, 2025. doi:10.3390/ai6080173.

[2] Manuel Bompard et al. Neural network construction, 2024. United States Patent US 12,124,779 B2.

[3] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *International Conference on Learning Representations (ICLR)*, 2019.

[4] D. Hughes, T. Haidinger, X. Pei, and C. Vagg. Real-time temperature prediction of electric machines using machine learning with physically informed features. *Energy and AI*, 2023.

[5] Wilhelm Kirchgässner, Oliver Wallscheid, and Joachim Böcker. Electric motor temperature dataset, 2021. Kaggle: https://www.kaggle.com/datasets/wkirgsn/electric-motor-temperature.

[6] Wilhelm Kirchgässner, Oliver Wallscheid, and Joachim Böcker. Thermal neural networks: Lumped-parameter thermal modeling with state-space machine learning. *Engineering Applications of Artificial Intelligence*, 2021.

[7] Yann LeCun, John S. Denker, and Sara A. Solla. Optimal brain damage. In *Advances in Neural Information Processing Systems (NeurIPS)*, 1990.

A Notation

Symbol	Meaning
L, n_ℓ	number of layers; width of layer ℓ
$\mathbf{W}^{(\ell)}, \boldsymbol{b}^{(\ell)}$	weight matrix and bias of layer ℓ
φ	hidden activation (tanh throughout)
P	total parameter count, eq. (4)
d	input dimension
J	sum-of-squares objective, eq. (11)
$\mathcal{L}$	constrained (Lagrangian) loss, eq. (12)
Λ_i	Lagrange multiplier for layer i's forward constraint
$\Delta\mathcal{L}$	cheap candidate saliency estimate, eq. (15)
Γ	network topology
$\varepsilon_a, \varepsilon_v, \tau, \rho$	additive/validation/subtractive/stopping thresholds
$\boldsymbol{g}, \mathbf{H}$	loss gradient and Hessian
$\mathbf{J}$	residual Jacobian, eq. (18)
μ	Levenberg–Marquardt damping
s_q, s_j	saliency of weight q / neuron j
$\boldsymbol{e}_q$	unit vector selecting coordinate q
Δ	dynamic-twin integration step
$x(t), \boldsymbol{u}(t)$	winding temperature and drive signals at time t

B Experimental configuration

Table 15: Key settings. Both tracks share splits, activation and error budget so that differences are attributable to method rather than protocol.

Setting	Value
Shared	
Hidden activation	tanh
Output layer	linear
Normalisation	z-score from training statistics only
Split order	split first, then normalise (no leakage)
Synthetic	
Samples	$N = 5000$, $\boldsymbol{x} \in [-3, 3]^d$
Noise (dataset B)	$\sigma = 12.5\,\%$ of peak-to-peak
Split	75 / 10 / 15 train/val/test
Baseline HPO	Optuna TPE, ≤ 240 trials
Baseline selection	smallest within 5 % of best val MSE
Baseline init	best-of-N restarts, selected on validation
PMSM	
Sampling rate	2 Hz
Global hold-out	14 profiles, never seen in search or selection
Development splits	5 profile-level splits
Feature set	11 raw / 81 with physics + EWMA (6 spans)
Dynamic twin stride	$\Delta = 5\,\mathrm{s}$
Pruning	
Sparsity sweep	$\{10, 20, \ldots, 90, 95\}\%$
Recovery	Adam phase + masked strong-Wolfe LBFGS polish
Selection	validation; reported on test/hold-out
Constructive	
Seed topology	$[d, 1, 1]$
Retraining	Gauss–Newton / Levenberg–Marquardt, multi-restart
ADD_LAYER scale	$\alpha = 0.3$ (SVD split)
Budget check	runs record `cycles_used`; truncated runs excluded

C Reproducibility

All results derive from a single repository with deterministic seeds. Points worth recording for anyone re-running the study:

- **Verify the search used its budget.** A capped cycle count produces a converged-*looking* result that is merely truncated, and such a result is indistinguishable from a genuine finding unless the budget is checked. Runs record whether they terminated on convergence or on the cap.
- **Baselines need multiple initialisations.** A single-seed baseline can land in a poor basin, which flatters every pruning method that happens to re-initialise and can produce apparent sub-baseline error ratios that are artefacts of the reference rather than achievements of the method.
- **Compare like with like.** Standardised and raw error scales must not be mixed between

tracks; all ratios in this report are raw-to-raw.

- **Profile-level splits.** Any sample-level shuffle on the motor data leaks and inflates accuracy substantially.

D Figure regeneration

Result figures are generated from stored checkpoints rather than redrawn by hand, so they stay consistent with the tables. One rendering detail is worth recording because it is easy to reintroduce: network diagrams must scale edge width and opacity *per layer*. Under a single global scale, a layer whose surviving weights are an order of magnitude smaller than another's falls below the visibility threshold, and a fully connected neuron can render as though it had no incoming connections at all — a picture that misrepresents the model.